

Tejas Rao: DATA241 | COMP241 Final Project

This project implements Zhang's camera calibration pipeline as described in:

Zhang Z. A flexible new technique for camera calibration. IEEE Transactions on pattern analysis and machine intelligence. 2000 Nov 30;22(11):1330-4.

This notebook uses slightly different notation from the original paper, and instead follows the Instructions PDF from this course. Some important notation has been summarised here, and symbols not included will be described during occurrence

Notation

- Homogeneous image point: $\mathbf{x} = (u, v, 1)^\top$.
- Pixel (non-homogeneous) image point: $\tilde{\mathbf{x}} = (u, v)^\top$.
- Planar world point on $Z = 0$: $\mathbf{X} = (X, Y, 1)^\top$.
- Intrinsic matrix: $\mathbf{K}$.
- Per-view homography: $\mathbf{H}_i = [\mathbf{h}_{1i} \ \mathbf{h}_{2i} \ \mathbf{h}_{3i}]$.
- IAC (image of absolute conic): $\boldsymbol{\omega} = (\mathbf{K}\mathbf{K}^\top)^{-1}$.
- A hat (for example, $\hat{\mathbf{H}}$) denotes an intermediate estimate before final normalisation/constraint enforcement.

Note: There were few things that I have worked on that weren't explicitly mentioned in the project document. These have been labelled with *(Extension)*

Index

- Stage 1: Data collection and corner detection
- Stage 2: Per-view homography via normalised DLT
- Stage 3: Linear solve for the IAC $\boldsymbol{\omega}$
- Stage 4: Intrinsic matrix recovery and OpenCV cross-check
- Stage 5: Extrinsic recovery and reprojection analysis
- Stage 6: View-count sensitivity, informed sampling comparison, and hypothesis testing

Setting up the notebook with imports and code formatter:

```
In [1]: # black formatter
        %load_ext jupyter_black

        # imports
        import numpy as np
```

```

import matplotlib.pyplot as plt
import cv2 as cv
import pandas as pd
import os
import re
from IPython.display import display, Math

# Global plotting style and save helper
PLOTS_DIR = os.path.join(os.getcwd(), "plots")
os.makedirs(PLOTS_DIR, exist_ok=True)

# Consistent serif typography + restrained colour palette across stages.
COLOUR_NAVY = "#1f3b73"
COLOUR_TEAL = "#2a9d8f"
COLOUR_CORAL = "#e76f51"
COLOUR_GOLD = "#e9c46a"
COLOUR_PLUM = "#6a4c93"
COLOUR_CRIMSON = "#c44536"

plt.rcParams.update(
    {
        "font.family": "serif",
        "font.serif": ["Times New Roman", "DejaVu Serif", "Liberation Ser
        "axes.titlesize": 13,
        "axes.labelsize": 11,
        "axes.titleweight": "semibold",
        "axes.prop_cycle": plt.cycler(
            color=[COLOUR_NAVY, COLOUR_TEAL, COLOUR_CORAL, COLOUR_PLUM, C
        ),
        "grid.alpha": 0.25,
        "grid.linestyle": "--",
    }
)

def _slugify_plot_text(text):
    text = str(text).strip().lower()
    text = re.sub(r"^[a-z0-9]+", "_", text)
    return text.strip("_")

def save_plot(section_name, brief_title, fig=None, dpi=300):
    fig = plt.gcf() if fig is None else fig
    filename = (
        f"_{slugify_plot_text(section_name)}___{slugify_plot_text(brief_t
    )
    out_path = os.path.join(PLOTS_DIR, filename)
    fig.savefig(out_path, dpi=dpi, bbox_inches="tight")
    print(f"Saved plot: {out_path}")

```

Stage 1: Data Collection and Corner Detection

In this stage, we aim to collect robust checkerboard correspondences

$$\{(\mathbf{X}_j, \tilde{\mathbf{x}}_{ij})\}_{j=1}^n$$

for multiple views i .

Stage 1.1: Demonstration

The code immediately below demonstrates detection, refinement, and visualisation of corners. To make the code more modular and enable corner detection for multiple images, they are wrapped in functions afterwards. These correspondences recovered then are the input for stage 2, i.e., homography estimation.

```
In [2]: # worked for initial image

# Load one reference view and detect checkerboard corners.
img = cv.imread("calibration_imgs/IMG_1353.JPG")
gray = cv.cvtColor(img, cv.COLOR_BGR2GRAY)

criteria = (cv.TERM_CRITERIA_EPS + cv.TERM_CRITERIA_MAX_ITER, 300, 0.0001)

ret, corners = cv.findChessboardCorners(gray, (9, 9))
corners2 = cv.cornerSubPix(gray, corners, (5, 5), (-1, -1), criteria)

# Draw the detected corners and show the overlay.
cv.drawChessboardCorners(img, (9, 9), corners2, ret)
# cv.imshow('img', img)
# cv.waitKey(0) # keep it open till key is pressed

plt.figure(figsize=(10, 8), dpi=300)
plt.imshow(img)
# plt.title(f"Chessboard Corners Detected in {image_path}")
plt.axis("off") # Hide axis ticks
save_plot("Stage 1.1", "Corner detection demo overlay")
plt.show()
# cv.destroyAllWindows()
```

Saved plot: /Users/tejasrao/Desktop/Master/Studies_at_Krea/Year 3/T9/CV/CalibrationProject/plots/stage_1_1__corner_detection_demo_overlay.png



Interpretation: The detected corner overlay shows that the chessboard detector is successfully detecting the corners in this view. Also serves as a sanity check

We show the correspondance between real world coordinates and pixel coordinates in the image plane using the code below:

```
In [3]: real_world_coord = [  
    (0, 0),  
    (0, 1),  
    (0, 2),  
    (0, 3),  
    (0, 4),  
    (0, 5),  
    (1, 0),  
    (1, 1),  
    (1, 2),  
    (1, 3),  
    (1, 4),  
    (1, 5),  
    (2, 0),  
    (2, 1),  
    (2, 2),  
    (2, 3),  
    (2, 4),  
    (2, 5),  
    (3, 0),  
    (3, 1),  
    (3, 2),  
    (3, 3),  
    (3, 4),  
    (3, 5),  
    (4, 0),  
    (4, 1),  
    (4, 2),
```

```

(4, 3),
(4, 4),
(4, 5),
(5, 0),
(5, 1),
(5, 2),
(5, 3),
(5, 4),
(5, 5),
(6, 0),
(6, 1),
(6, 2),
(6, 3),
(6, 4),
(6, 5),
(7, 0),
(7, 1),
(7, 2),
(7, 3),
(7, 4),
(7, 5),
(8, 0),
(8, 1),
(8, 2),
(8, 3),
(8, 4),
(8, 5),
]

# Overlay the detected image points to visualise point ordering.
img = cv.imread("calibration_imgs/IMG_1353.JPG")
plt.figure(figsize=(10, 8))
plt.imshow(img)

for i in range(len(real_world_coord)):
    plt.scatter(corners2[i][0][0], corners2[i][0][1], c=COLOUR_CORAL, s=2)

plt.axis("off")
save_plot("Stage 1.1", "Detected corner ordering overlay")
plt.show()

```

Saved plot: /Users/tejasrao/Desktop/Master/Studies_at_Krea/Year 3/T9/CV/CalibrationProject/plots/stage_1_1__detected_corner_ordering_overlay.png



Interpretation: The scattered image points illustrate the correspondence order used in the notebook. A stable, row-major ordering is important because homography estimation assumes each world point $\mathbf{X}_j$ is paired with the matching image point $\tilde{\mathbf{x}}_{ij}$.

Stage 1.2: Functionalised code

The next block just wraps the above code into a function, facilitating modular and scalable correspondence extraction for all images.

For each image i , the `get_corners` function, does the following:

1. Detect checkerboard corners (OpenCV black-box).
2. Refine to sub-pixel coordinates (Black Box)
3. Return corners in a consistent ordering/shape for later matrix construction.

Mathematically, detections provide

$$\tilde{\mathbf{x}}_{ij} = (u_{ij}, v_{ij})^T \implies \mathbf{x}_{ij} = (u_{ij}, v_{ij}, 1)^T.$$

Combined with known board coordinates $\mathbf{X}_j$, this yields per-view correspondence sets. These sets will be normalised for DLT in the subsequent stages.

I also define other useful functions that help convert euclidean coordinates to homogeneous coordinates (`to_hc`), and a function to normalise the homogeneous coordinates (`norm`)

```
In [4]: # utils
to_hc = lambda x: np.array([*x, 1])
norm = lambda x: x / x[-1]

CHESSBOARD_SIZE = (9, 9) # as board is 10 x 10 squares
```

```

def get_corners(path, show=False):
    img = cv.imread(path)
    if img is None:
        raise FileNotFoundError(f"Could not read image: {path}")

    gray = cv.cvtColor(img, cv.COLOR_BGR2GRAY)

    # Robust detector first
    ret, corners = cv.findChessboardCornersSB(gray, CHESSBOARD_SIZE, None)

    # Fallback to classic with helpful flags
    if not ret or corners is None:
        flags = cv.CALIB_CB_ADAPTIVE_THRESH + cv.CALIB_CB_NORMALIZE_IMAGE
        ret, corners = cv.findChessboardCorners(gray, CHESSBOARD_SIZE, fl

    if not ret or corners is None:
        raise RuntimeError(f"Chessboard corners not found in: {path}")

    # Sub-pixel refinement (for classic or SB output alike)
    criteria = (cv.TERM_CRITERIA_EPS + cv.TERM_CRITERIA_MAX_ITER, 300, 1e
    corners_refined = cv.cornerSubPix(gray, corners, (5, 5), (-1, -1), cr

    if show:
        vis = img.copy()
        cv.drawChessboardCorners(vis, CHESSBOARD_SIZE, corners_refined, r
        plt.figure(figsize=(8, 6), dpi=150)
        plt.imshow(cv.cvtColor(vis, cv.COLOR_BGR2RGB))
        plt.axis("off")
        save_plot("Stage 1.2", "Single-image corner extraction preview")
        plt.show()

    return corners_refined

```

Stage 1.3: Deliverable

As per the project outline, the following are expected:

- a **3 x 3 grid** of representative input images with detected corners overlaid,
- a per-image corner detection table (`num_corners_found` , `detection_success`),
- overall detection success rate.

The code below now reports the table **and** renders the required 3 x 3 corner-overlay grid.

```

In [5]: # Read in images
candidate_roots = ["calibration_imgs", "calibration_imgs"]
stage1_images = []
for rr in candidate_roots:
    if os.path.isdir(rr):
        stage1_images = sorted(
            [f for f in os.listdir(rr) if f.lower().endswith((".jpg", ".j
        )
        if stage1_images:
            stage1_root = rr
            break
stage1_rows = []
stage1_success_images = []

```

```

stage1_corners_by_image = {}

# Detect corners once per image and keep them for later stages.
for image_name in stage1_images:
    image_path = os.path.join(stage1_root, image_name)
    try:
        corners_stage1 = get_corners(image_path)
        corners_stage1 = corners_stage1.reshape(-1, 1, 2)
        n_found = int(corners_stage1.shape[0])
        success = True
        stage1_success_images.append(image_name)
        stage1_corners_by_image[image_name] = corners_stage1
    except Exception:
        n_found = 0
        success = False
    stage1_rows.append(
        {
            "image": image_name,
            "num_corners_found": n_found,
            "detection_success": success,
        }
    )

stage1_detection_table = pd.DataFrame(stage1_rows) # deliverable table

print(
    f"Stage 1 success rate: {stage1_detection_table['detection_success'].sum() / len(stage1_detection_table)}"
)
display(stage1_detection_table)

# Build a representative 3x3 corner-overlay panel.
rep_idx = np.linspace(0, len(stage1_success_images) - 1, 9, dtype=int)
representative_images = [stage1_success_images[k] for k in rep_idx]
fig, axes = plt.subplots(3, 3, figsize=(12, 12), dpi=300)
axes = axes.flatten()
for ax in axes:
    ax.axis("off")
for ax, image_name in zip(axes, representative_images):
    image_path = os.path.join(stage1_root, image_name)
    image_bgr = cv.imread(image_path)
    vis = image_bgr.copy()
    corners = stage1_corners_by_image[image_name]
    cv.drawChessboardCorners(vis, CHESSBOARD_SIZE, corners, True)
    # Add clearer corner markers for display readability.
    for corner in corners:
        x, y = corner.ravel()
        cv.circle(vis, (int(x), int(y)), 15, (0, 0, 255), 2)
    ax.imshow(cv.cvtColor(vis, cv.COLOR_BGR2RGB))
    ax.set_title(image_name, fontsize=10)
    ax.axis("off")
plt.suptitle("Stage 1 Deliverable: 3x3 Representative Corner Detection Grid")
plt.tight_layout()
save_plot("Stage 1 Deliverable", "3x3 representative corner detection grid")
plt.show()

```

```

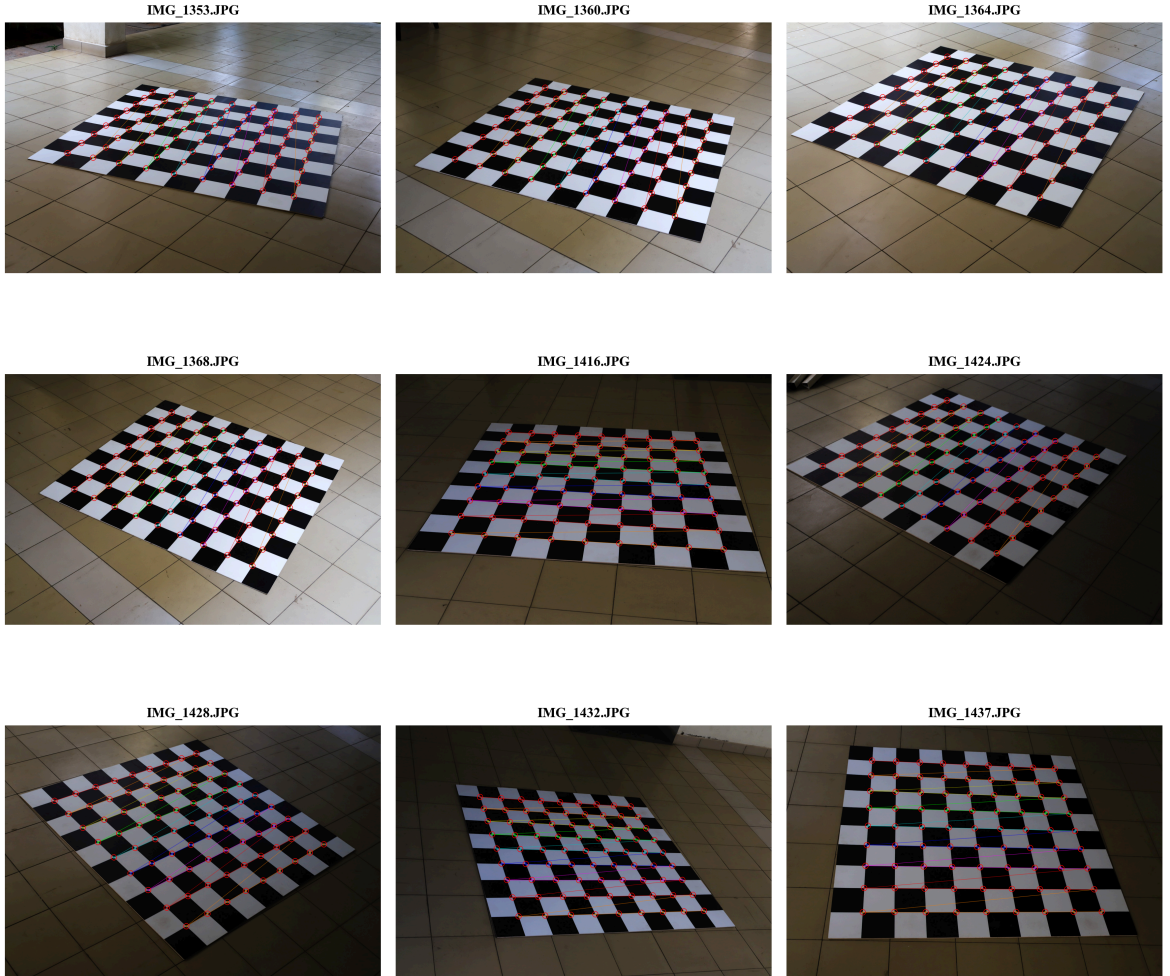
[ERROR:1@5.980] global ocl.cpp:4679 createFromBinary OpenCL error CL_INVALID_VALUE (-30) during call: clCreateProgramWithBinary
[ERROR:0@6.192] global ocl.cpp:4679 createFromBinary OpenCL error CL_INVALID_VALUE (-30) during call: clCreateProgramWithBinary
Stage 1 success rate: 93.33%

```

	image	num_corners_found	detection_success
0	IMG_1353.JPG	81	True
1	IMG_1354.JPG	81	True
2	IMG_1355.JPG	81	True
3	IMG_1360.JPG	81	True
4	IMG_1362.JPG	81	True
5	IMG_1363.JPG	81	True
6	IMG_1364.JPG	81	True
7	IMG_1365.JPG	81	True
8	IMG_1366.JPG	81	True
9	IMG_1367.JPG	81	True
10	IMG_1368.JPG	81	True
11	IMG_1369.JPG	81	True
12	IMG_1372.JPG	81	True
13	IMG_1415.JPG	0	False
14	IMG_1416.JPG	81	True
15	IMG_1418.JPG	81	True
16	IMG_1422.JPG	81	True
17	IMG_1424.JPG	81	True
18	IMG_1425.JPG	81	True
19	IMG_1426.JPG	81	True
20	IMG_1427.JPG	81	True
21	IMG_1428.JPG	81	True
22	IMG_1429.JPG	81	True
23	IMG_1430.JPG	81	True
24	IMG_1431.JPG	0	False
25	IMG_1432.JPG	81	True
26	IMG_1433.JPG	81	True
27	IMG_1434.JPG	81	True
28	IMG_1436.JPG	81	True
29	IMG_1437.JPG	81	True

Saved plot: /Users/tejasrao/Desktop/Master/Studies_at_Krea/Year 3/T9/CV/C
alibrationProject/plots/stage_1_deliverable__3x3_representative_corner_de
tection_grid.png

Stage 1 Deliverable: 3x3 Representative Corner Detection Grid



Stage 2: Per-View Homography via Normalised DLT

Setup of Normalised DLT Formulation

For each view i , we aim estimate

$$\mathbf{x}_{ij} \sim \mathbf{H}_i \mathbf{X}_j.$$

To improve numerical conditioning, we apply point normalisation:

$$\bar{\mathbf{x}}_{ij} = \mathbf{T}_{\tilde{x},i} \mathbf{x}_{ij}, \quad \bar{\mathbf{X}}_j = \mathbf{T}_X \mathbf{X}_j,$$

with centroid at the origin and average radial distance $\sqrt{2}$.

Then we solve for a normalised homography and denormalise to recover $\mathbf{H}_i$.

To normalise the image points for a more stable homography estimation, we need the two similarity transforms:

- image-point transform $\mathbf{T}_{\tilde{x},i}$ (often called $\mathbf{T}$),
- world-plane transform $\mathbf{T}_X$ (often called $\mathbf{T}'$).

Stage 2.1: To compute $\mathbf{T}$ and $\mathbf{T}'$

Take any 2D point set in homogeneous form $\mathbf{p}_k = (p_{k1}, p_{k2}, w_k)^\top$, and convert to non-homogeneous points $\tilde{\mathbf{p}}_k = \left(\frac{p_{k1}}{w_k}, \frac{p_{k2}}{w_k} \right)^\top$.

Then, compute centroid $\bar{\mathbf{p}} = \frac{1}{N} \sum_{k=1}^N \tilde{\mathbf{p}}_k$,

The mean radial distance is $\bar{d} = \frac{1}{N} \sum_{k=1}^N \|\tilde{\mathbf{p}}_k - \bar{\mathbf{p}}\|_2$, and the given scale is $s = \frac{\sqrt{2}}{\bar{d}}$

Then the normalisation matrix is

$$\mathbf{T}(\bar{\mathbf{p}}, s) = \begin{bmatrix} s & 0 & -s\bar{p}_1 \\ 0 & s & -s\bar{p}_2 \\ 0 & 0 & 1 \end{bmatrix}.$$

If we apply this procedure for image points $\mathbf{x}_{ij}$, we obtain $\mathbf{T}_{\tilde{x},i}$. Similarly, if we apply this procedure for world planar points $\mathbf{X}_j$, we can derive $\mathbf{T}_X$.

In this notebook:

- `Tu` corresponds to $\mathbf{T}_{\tilde{x},i}$,
- `TX` corresponds to $\mathbf{T}_X$.

The function `compute_normalisation_transform` in the cell below does the same thing.

```
In [6]: def compute_normalization_transform(points):
        """
        Accepts either:
        - inhomogeneous points: (N, 2)
        - homogeneous points:   (N, 3)

        Returns T such that normalised points have centroid at origin
        and mean distance sqrt(2).
        """
        points = np.asarray(points, dtype=np.float64)

        # Normalise strictly from 2D coordinates (x, y), even when input is h
        if points.ndim != 2 or points.shape[1] not in (2, 3):
            raise ValueError(f"Expected (N,2) or (N,3) points, got shape {poi

        if points.shape[1] == 3:
            # Convert homogeneous -> inhomogeneous safely.
            w = points[:, 2:3]
            if np.any(np.isclose(w, 0.0)):
                raise ValueError("Homogeneous point with w=0 encountered")
            pts2 = points[:, :2] / w
        else:
            pts2 = points

        centroid = pts2.mean(axis=0)
        shifted = pts2 - centroid
        d = np.sqrt(np.sum(shifted * shifted, axis=1))
        mean_dist = np.mean(d)

        scale = np.sqrt(2.0) / mean_dist # scaling
```

```

T = np.array(
    [
        [scale, 0.0, -scale * centroid[0]],
        [0.0, scale, -scale * centroid[1]],
        [0.0, 0.0, 1.0],
    ],
    dtype=np.float64,
)

return T, centroid, scale

```

To extract the normalised points using the `compute_normalisation_transform` function, we use the following code. This will be useful when we use the normalised points to calculate the homography

```

In [7]: # util function that will help calculate normalised points
normalise_with_transform = lambda T, points_hc: [T @ pt for pt in points_

def normalised_points(points):
    T, _, _ = compute_normalization_transform(points)
    return normalise_with_transform(T, points_hc=points)

```

Stage 2.2: Design Matrix Construction

The function `Homography_definition_Normalised` (below) uses the following method to construct the Design matrix used to derive the homography.

For one view i (after point normalisation), define

$$\bar{\mathbf{X}}_j = \begin{bmatrix} \bar{X}_j \\ \bar{Y}_j \\ 1 \end{bmatrix}, \quad \bar{\mathbf{x}}_{ij} = \begin{bmatrix} \bar{u}_{ij} \\ \bar{v}_{ij} \\ 1 \end{bmatrix}.$$

Let

$$\hat{\mathbf{H}}_i = \begin{bmatrix} \hat{h}_{11} & \hat{h}_{12} & \hat{h}_{13} \\ \hat{h}_{21} & \hat{h}_{22} & \hat{h}_{23} \\ \hat{h}_{31} & \hat{h}_{32} & \hat{h}_{33} \end{bmatrix}, \quad \hat{\mathbf{h}}_i = [\hat{h}_{11} \quad \hat{h}_{12} \quad \hat{h}_{13} \quad \hat{h}_{21} \quad \hat{h}_{22} \quad \hat{h}_{23} \quad \hat{h}_{31} \quad \hat{h}_{32} \quad \hat{h}_{33}]^\top.$$

From

$$\bar{\mathbf{x}}_{ij} \times (\hat{\mathbf{H}}_i \bar{\mathbf{X}}_j) = \mathbf{0},$$

use two independent linear equations per correspondence j :

$$\mathbf{a}_{ij}^{(1)\top} = [\bar{X}_j \quad \bar{Y}_j \quad 1 \quad 0 \quad 0 \quad 0 \quad -\bar{u}_{ij}\bar{X}_j \quad -\bar{u}_{ij}\bar{Y}_j \quad -\bar{u}_{ij}],$$

$$\mathbf{a}_{ij}^{(2)\top} = [0 \quad 0 \quad 0 \quad \bar{X}_j \quad \bar{Y}_j \quad 1 \quad -\bar{v}_{ij}\bar{X}_j \quad -\bar{v}_{ij}\bar{Y}_j \quad -\bar{v}_{ij}].$$

Stack rows over all n corners:

$$\mathbf{A}_i = \begin{bmatrix} \mathbf{a}_{i1}^{(1)\top} \\ \mathbf{a}_{i1}^{(2)\top} \\ \vdots \\ \mathbf{a}_{in}^{(1)\top} \\ \mathbf{a}_{in}^{(2)\top} \end{bmatrix} \in \mathbb{R}^{2n \times 9}, \quad \mathbf{A}_i \hat{\mathbf{h}}_i = \mathbf{0}.$$

SVD gives $\hat{\mathbf{h}}_i$ as the last right-singular vector, reshaped to the normalised homography $\hat{\mathbf{H}}_i$.

Then $\hat{\mathbf{H}}_i$ denormalised to $\mathbf{H}_i$ in the following method:

$$\mathbf{H}_i = \mathbf{T}_{\tilde{x},i}^{-1} \hat{\mathbf{H}}_i \mathbf{T}_X.$$

Since $\mathbf{A}_i \in \mathbb{R}^{2n \times 9}$, we need at least $2n \geq 8 \Rightarrow n \geq 4$ correspondences.

Note: As we are required to calculate the mean reprojection error for each view (deliverable), the cell below also contains a helper function (`reprojection_error`) for the same. For a view i , the mean reprojection error e_i is given by: $\bar{e}_i = \frac{1}{n} \sum_{j=1}^n \|\tilde{\mathbf{x}}_{ij} - \pi(\mathbf{H}_i \mathbf{X}_j)\|_2$

```
In [8]: # Utils
def reprojection_error(Homography, corners_hc, real_world_hc):

    rep_all = []
    for i in range(len(corners_hc)):
        rep_error_vec = np.abs(corners_hc[i] - norm(Homography @ real_wor
        rep_error = (rep_error_vec**2).sum()

        rep_all.append(rep_error)

    mean_rep_error = np.array(rep_all).mean()

    return mean_rep_error

# Normalised DLT implementation to derive normalised H
def Homography_definition_Normalised(path):

    corners = get_corners(path)

    # Match world-point grid to detected checkerboard inner-corner size.
    n_cols, n_rows = CHESSBOARD_SIZE
    real_world_coord = [(x, y) for y in range(n_rows) for x in range(n_co

    corners_hc = []
    real_world_hc = []

    for corner in corners:
        corners_hc.append(to_hc(corner[0]))
    for real_world_pt in real_world_coord:
        real_world_hc.append(to_hc(real_world_pt))

    corners_hc_raw = np.asarray(corners_hc, dtype=np.float64)
    real_world_hc_raw = np.asarray(real_world_hc, dtype=np.float64)

    if len(corners_hc_raw) != len(real_world_hc_raw):
        raise ValueError(
```

```

        f"Mismatch: corners={len(corners_hc_raw)}, world={len(real_wo
        f"Set CHESSBOARD_SIZE correctly."
    )

    # Use world transform for world points and image transform for image
    Tx, _, _ = compute_normalization_transform(real_world_hc_raw)
    Tu, _, _ = compute_normalization_transform(corners_hc_raw)

    real_world_hc_n = np.asarray(
        [Tx @ pt for pt in real_world_hc_raw], dtype=np.float64
    )
    corners_hc_n = np.asarray([Tu @ pt for pt in corners_hc_raw], dtype=n

    L_mat = []

    for i in range(len(real_world_hc_n)):
        X = real_world_hc_n[i]
        u = corners_hc_n[i]
        row1 = [*X, 0, 0, 0, -X[0] * u[0], -X[1] * u[0], -u[0]]
        row2 = [0, 0, 0, *X, -X[0] * u[1], -X[1] * u[1], -u[1]]
        L_mat.append(row1)
        L_mat.append(row2)

    L_mat = np.asarray(L_mat, dtype=np.float64)

    # DLT in normalised space
    _, _, Vt = np.linalg.svd(L_mat)
    v_min = Vt[-1]
    H_hat = v_min.reshape(3, 3)

    H = np.linalg.inv(Tu) @ H_hat @ Tx

    # Normalise homography scale
    H = H / H[2, 2]

    # Compute reprojection error in original (unnormalised) coordinates.
    rep_error = reprojection_error(H, corners_hc_raw, real_world_hc_raw)

    return H, rep_error

# EXAMPLE CALL
# H_norm, rep_error = Homography_definition_Normalised("calibration_imgs/

```

Stage 2.3: Deriving Homography

The below code uses all valid images to get the homography.

```

In [9]: root = "calibration_imgs/" # or calibration_imgs/ (check exact folder na
images_list = sorted(
    [f for f in os.listdir(root) if f.lower().endswith((".jpg", ".jpeg",
)

H_i_all = []
mean_e_i_all = []
images_list_filtered = []
failed_images = []

```

```

for img_name in images_list:
    path = os.path.join(root, img_name)
    try:
        Hn, repn = Homography_definition_Normalised(path)
        H_i_all.append(Hn)
        mean_e_i_all.append(repn)
        images_list_filtered.append(img_name)
    except RuntimeError as e:
        failed_images.append((img_name, str(e)))

print(f"Used {len(images_list_filtered)} images, skipped {len(failed_images)}")
print("Failed:", [x[0] for x in failed_images[:10]])

```

Used 28 images, skipped 2
Failed: ['IMG_1415.JPG', 'IMG_1431.JPG']

Stage 2.4: Filtering according to Reprojection error per view (Deliverable).

We are required to rejecting views with high homography reprojection error. The threshold is $\bar{e}_i \leq 1.5$ px.

The code below creates the per-view error table and filters the image list used by later stages. However, from the dataset used for this project, we see that no view has a reprojection error ≥ 1.5 px, meaning all views are used in subsequent stages.

This code also displays the first deliverable for stage 2, i.e., table with mean reprojection error per view

```

In [10]: dict_stage2 = {
    "images": images_list_filtered,
    "Reprojection Error (Normalised)": mean_e_i_all,
}

df_rep_error_perView = pd.DataFrame(dict_stage2)
df_rep_error_perView = df_rep_error_perView.loc[
    df_rep_error_perView["Reprojection Error (Normalised)"] <= 1.5
]
images_list_filtered = list(df_rep_error_perView.images)

# Deliverable
display(df_rep_error_perView)

```

	images	Reprojection Error (Normalised)
0	IMG_1353.JPG	0.249686
1	IMG_1354.JPG	0.374952
2	IMG_1355.JPG	0.319798
3	IMG_1360.JPG	0.216337
4	IMG_1362.JPG	0.247811
5	IMG_1363.JPG	0.202417
6	IMG_1364.JPG	0.261394
7	IMG_1365.JPG	0.307208
8	IMG_1366.JPG	0.272737
9	IMG_1367.JPG	0.186871
10	IMG_1368.JPG	0.140122
11	IMG_1369.JPG	0.228999
12	IMG_1372.JPG	0.204906
13	IMG_1416.JPG	0.268850
14	IMG_1418.JPG	0.166385
15	IMG_1422.JPG	0.197518
16	IMG_1424.JPG	0.190869
17	IMG_1425.JPG	0.271853
18	IMG_1426.JPG	0.235211
19	IMG_1427.JPG	0.173969
20	IMG_1428.JPG	0.167176
21	IMG_1429.JPG	0.162644
22	IMG_1430.JPG	0.166806
23	IMG_1432.JPG	0.167261
24	IMG_1433.JPG	0.196276
25	IMG_1434.JPG	0.188523
26	IMG_1436.JPG	0.240144
27	IMG_1437.JPG	0.260069

Stage 2.4: Overlay Deliverable

We visualise two representative kept views:

- the view with minimum $\bar{e}_i$,
- the view with maximum $\bar{e}_i$.

For each selected view, overlay detected corners $\tilde{\mathbf{x}}_{ij}$ and homography reprojected corners $\hat{\mathbf{x}}_{ij} = \pi(\mathbf{H}_i \mathbf{X}_j)$.

```
In [11]: # Map each successful image to its precomputed homography H_i
H_i_by_image = {img: H_i for img, H_i in zip(dict_stage2["images"], H_i_a

# define best and worst image by mean reprojected error
best_image = df_rep_error_perView.loc[
    df_rep_error_perView["Reprojection Error (Normalised)"].idxmin(), "im
]
worst_image = df_rep_error_perView.loc[
    df_rep_error_perView["Reprojection Error (Normalised)"].idxmax(), "im
]
selected_images = [best_image, worst_image]

n_cols, n_rows = CHESSBOARD_SIZE
X_world_h = np.array(
    [to_hc((x, y)) for y in range(n_rows) for x in range(n_cols)], dtype=
) # equivalent to real_world_coords

# Plot detected points against homography reprojections on two views.
fig, axes = plt.subplots(1, 2, figsize=(14, 6))

for ax, image_name in zip(axes, selected_images):
    image_path = os.path.join(root, image_name)
    H_i = H_i_by_image[image_name]

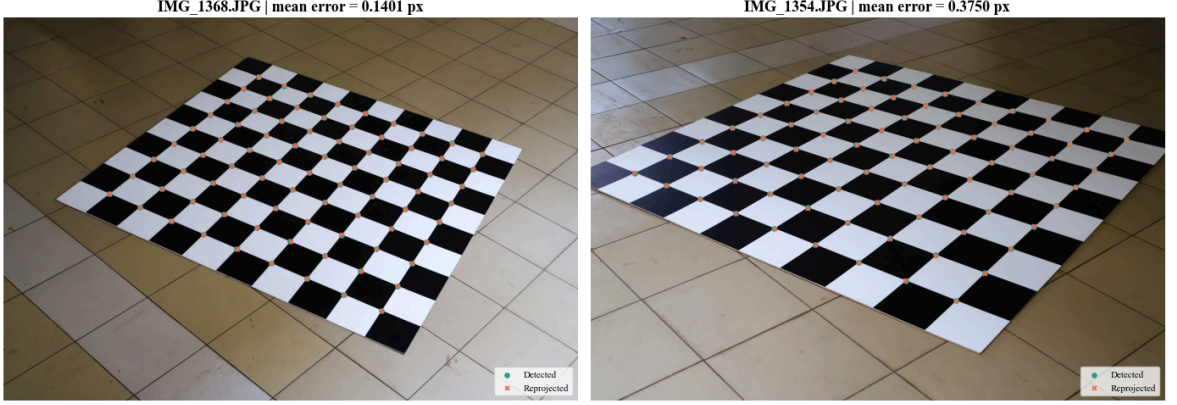
    # Collect detected corners and projected corners for this view.
    x_detected = get_corners(image_path).reshape(-1, 2).astype(np.float64)
    x_hat_h = (H_i @ X_world_h.T).T
    x_reproj = x_hat_h[:, :2] / x_hat_h[:, [2]]

    image_rgb = cv.cvtColor(cv.imread(image_path), cv.COLOR_BGR2RGB)
    ax.imshow(image_rgb)
    ax.scatter(
        x_detected[:, 0], x_detected[:, 1], s=14, c=COLOUR_TEAL, label="D
    )
    ax.scatter(
        x_reproj[:, 0],
        x_reproj[:, 1],
        s=12,
        c=COLOUR_CORAL,
        marker="x",
        label="Reprojected",
    )

    e_i = float(
        df_rep_error_perView.loc[
            df_rep_error_perView["images"] == image_name,
            "Reprojection Error (Normalised)",
        ].iloc[0]
    )
    ax.set_title(f"{image_name} | mean error = {e_i:.4f} px")
    ax.axis("off")
    ax.legend(loc="lower right", fontsize=8)

plt.tight_layout()
save_plot("Stage 2.4", "Best and worst view homography overlays", fig=fig)
plt.show()
```

Saved plot: /Users/tejasrao/Desktop/Master/Studies_at_Krea/Year 3/T9/CV/C
alibrationProject/plots/stage_2_4__best_and_worst_view_homography_overlay
s.png



Stage 3: Linear Solve for the IAC ω

Stage 3.1: Derive matrix $\mathbf{L}$

For each view i , write

$$\mathbf{H}_i = [\mathbf{h}_{1i} \ \mathbf{h}_{2i} \ \mathbf{h}_{3i}], \quad \mathbf{h}_{ki} = \begin{bmatrix} h_{1k}^{(i)} & h_{2k}^{(i)} & h_{3k}^{(i)} \end{bmatrix}^\top.$$

Using rotation orthonormality, we get two constraints per view:

$$\mathbf{h}_{1i}^\top \omega \mathbf{h}_{2i} = 0, \quad \mathbf{h}_{1i}^\top \omega \mathbf{h}_{1i} - \mathbf{h}_{2i}^\top \omega \mathbf{h}_{2i} = 0.$$

Following Zhang's Paper, we can define the 6-vector

$$\mathbf{v}_{pq}^{(i)} = \begin{bmatrix} h_{1p}^{(i)} h_{1q}^{(i)}, & h_{1p}^{(i)} h_{2q}^{(i)} + h_{2p}^{(i)} h_{1q}^{(i)}, & h_{2p}^{(i)} h_{2q}^{(i)}, & h_{3p}^{(i)} h_{1q}^{(i)} + h_{1p}^{(i)} h_{3q}^{(i)}, & h_{3p}^{(i)} h_{2q}^{(i)} + h_{2p}^{(i)} h_{3q}^{(i)}, & h_{3p}^{(i)} h_{3q}^{(i)} \end{bmatrix}.$$

The code `construct_L_rows_from_H_i` builds exactly two rows per view:

$$\boldsymbol{\ell}_{i,1}^\top = \mathbf{v}_{12}^{(i)\top}, \quad \boldsymbol{\ell}_{i,2}^\top = \left(\mathbf{v}_{11}^{(i)} - \mathbf{v}_{22}^{(i)} \right)^\top.$$

Then `construct_L_matrix` stacks all views:

$$\mathbf{L} = \begin{bmatrix} \boldsymbol{\ell}_{1,1}^\top \\ \boldsymbol{\ell}_{1,2}^\top \\ \vdots \\ \boldsymbol{\ell}_{m,1}^\top \\ \boldsymbol{\ell}_{m,2}^\top \end{bmatrix} \in \mathbb{R}^{2m \times 6},$$

where m is the number of retained views.

Finally, solve $\mathbf{L}\mathbf{b}_\omega = \mathbf{0}$ by SVD.

Since $\mathbf{b}_\omega \in \mathbb{R}^6$ and each view gives 2 rows, for full calibration of $\mathbf{K}$ via ω : at least **3 views** are needed (2 IAC constraints per view $\times$ 3 views).

Stage 3.1: Reducing Each Constraint to a Row of the Form $\mathbf{v}^\top \mathbf{b}_\omega = 0$

Let

$$\mathbf{b}_\omega = [\omega_{11} \quad \omega_{12} \quad \omega_{22} \quad \omega_{13} \quad \omega_{23} \quad \omega_{33}]^\top,$$

and write homography columns as

$$\mathbf{h}_p = [h_{1p} \quad h_{2p} \quad h_{3p}]^\top, \quad \mathbf{h}_q = [h_{1q} \quad h_{2q} \quad h_{3q}]^\top.$$

Using

$$\boldsymbol{\omega} = \begin{bmatrix} \omega_{11} & \omega_{12} & \omega_{13} \\ \omega_{12} & \omega_{22} & \omega_{23} \\ \omega_{13} & \omega_{23} & \omega_{33} \end{bmatrix},$$

we expand $\mathbf{h}^\top \boldsymbol{\omega} \mathbf{h}_q = (h_{1p}h_{1q} + h_{2p}h_{2q} + h_{3p}h_{3q})\omega_{11} +$

- $(h_{1p}h_{2q} + h_{2p}h_{1q})\omega_{12} +$
- $(h_{2p}h_{2q})\omega_{22} +$
- $(h_{3p}h_{1q} + h_{1p}h_{3q})\omega_{13} +$
- $(h_{3p}h_{2q} + h_{2p}h_{3q})\omega_{23} +$
- $(h_{3p}h_{3q})\omega_{33}.$

Define

$\mathbf{v}^{pq\top} = \begin{bmatrix} h_{1p}h_{1q}, & h_{1p}h_{2q} + h_{2p}h_{1q}, \\ h_{2p}h_{2q}, & h_{3p}h_{1q} + h_{1p}h_{3q}, \\ h_{3p}h_{2q} + h_{2p}h_{3q}, & h_{3p}h_{3q} \end{bmatrix}^\top,$

so

$\mathbf{h}^\top \boldsymbol{\omega} \mathbf{h}_q = \mathbf{v}^{pq\top} \mathbf{b}_\omega.$

Therefore, for each view i , Zhang's two constraints become linear rows in $\mathbf{L}$:

$$\mathbf{h}_{1i}^\top \boldsymbol{\omega} \mathbf{h}_{2i} = 0 \iff \mathbf{v}_{12}^{(i)\top} \mathbf{b}_\omega = 0,$$

$$\mathbf{h}_{1i}^\top \boldsymbol{\omega} \mathbf{h}_{1i} - \mathbf{h}_{2i}^\top \boldsymbol{\omega} \mathbf{h}_{2i} = 0 \iff (\mathbf{v}_{11}^{(i)} - \mathbf{v}_{22}^{(i)})^\top \mathbf{b}_\omega = 0.$$

```
In [12]: def construct_L_rows_from_H_i(H_i):
v = []
for i, j in [(0, 0), (0, 1), (1, 1)]:
    v_ij = np.array(
        [
            H_i[0][i] * H_i[0][j],
            H_i[0][i] * H_i[1][j] + H_i[1][i] * H_i[0][j],
            H_i[1][i] * H_i[1][j],
            H_i[2][i] * H_i[0][j] + H_i[0][i] * H_i[2][j],
            H_i[2][i] * H_i[1][j] + H_i[1][i] * H_i[2][j],
            H_i[2][i] * H_i[2][j],
        ]
    )
```

```

        ]
    )

    v.append(v_ij)

    l_row_1 = v[1]
    l_row_2 = v[0] - v[2]

    L_rows_from_H_i = np.array([l_row_1, l_row_2])

    return L_rows_from_H_i

def construct_L_matrix(H_i_list, b_omega=None, print_vtb=False):
    if len(H_i_list) == 0:
        raise ValueError("H_i_list is empty")

    L_rows_list = []
    for H_i in H_i_list:
        L_rows_list.append(construct_L_rows_from_H_i(H_i))

    L = np.vstack(L_rows_list)

    # Optional check: each row is a  $v^T$ , so row dot b should be  $\sim 0$ .
    if print_vtb:
        if b_omega is None:
            raise ValueError("Pass b_omega when print_vtb=True")
        vtb = L @ b_omega
        print("Row-wise check ( $v^T b$ ):")
        print(vtb)
        print("max  $|v^T b|$ :", float(np.max(np.abs(vtb))))

    return L

L = construct_L_matrix(H_i_all)
L

```

```
Out[12]: array([[ 1.27014426e+04, -1.49125045e+03, -3.90243564e+02,
 4.92820922e+00,  1.11021006e+00, -6.50171634e-04],
 [-8.35664999e+03, -8.90556138e+03,  2.66039254e+02,
 1.26978163e+01, -1.74266898e+00,  2.03563419e-03],
 [ 1.45661108e+04, -1.19630590e+02, -1.05585626e+03,
 4.46856160e-01,  2.22740798e+00, -1.17295658e-03],
 [ 7.58391713e+03, -1.62738027e+04,  6.16564628e+02,
 1.73424697e+01, -1.48054942e+00,  8.76719966e-04],
 [-6.19467605e+03,  3.77051246e+03,  1.21452958e+02,
 -5.23221189e+00, -2.03001396e-01,  5.42902122e-05],
 [ 1.46196416e+04,  3.94088317e+03, -9.12715334e+02,
 -4.23057826e+00,  2.47271756e+00, -1.65851086e-03],
 [ 1.25567200e+04, -1.91459185e+03, -7.78886832e+02,
 4.34629853e+00,  1.52328811e+00, -6.33354346e-04],
 [-6.88450310e+03, -1.25152616e+04,  5.27093077e+02,
 1.23834918e+01, -2.27488676e+00,  1.79591852e-03],
 [ 9.86313600e+03,  1.83674249e+03, -9.90297556e+02,
 -1.30465189e+00,  1.90521683e+00, -9.11369510e-04],
 [ 1.10303993e+04, -1.28742945e+04, -9.12493089e+01,
 1.26026184e+01, -1.05765931e-01,  1.85717266e-04],
 [-2.57276827e+03,  6.12607819e+03, -6.09895136e+01,
 -5.26815000e+00,  3.44068306e-01, -2.46791842e-04],
 [ 1.83764641e+04,  1.30140114e+03, -1.98502688e+03,
 3.83554947e-01,  3.33333685e+00, -1.37006079e-03],
 [ 1.29234199e+04, -1.73784065e+03, -1.12552649e+03,
 2.17807098e+00,  1.94368753e+00, -8.30703639e-04],
 [ 1.59026351e+02, -1.56681111e+04,  1.06731258e+03,
 1.38380908e+01, -2.27466300e+00,  1.17633536e-03],
 [ 1.16845792e+04,  1.31108442e+03, -1.26002753e+03,
 -6.47664537e-01,  2.22419539e+00, -9.76138615e-04],
 [ 9.76589647e+03, -1.57796762e+04,  1.67832707e+02,
 1.41669574e+01, -6.26830873e-01,  4.23220264e-04],
 [-5.59111028e+03,  4.50550186e+03,  2.97860552e+02,
 -4.77723299e+00, -3.11709237e-01,  1.68637552e-05],
 [ 1.40244434e+04,  5.35629912e+03, -1.41100407e+03,
 -3.46892370e+00,  2.90411316e+00, -1.43968249e-03],
 [ 1.24345640e+04,  5.71858938e+02, -1.39171573e+03,
 2.13759125e+00,  2.22371427e+00, -7.53578888e-04],
 [ 1.10023528e+03, -1.66448685e+04, -2.59602482e+02,
 1.31711723e+01, -1.32457763e+00,  1.19878794e-03],
 [ 9.78310997e+03, -1.07483009e+03, -1.64635145e+03,
 1.93719241e+00,  1.88583009e+00, -4.96188047e-04],
 [-9.22851345e+02, -1.61130358e+04,  7.29834401e+02,
 9.45496072e+00, -1.93679951e+00,  8.89301180e-04],
 [-2.38287131e+03,  8.42031116e+03,  3.97063032e+02,
 -4.14470806e+00,  2.69478984e-01, -2.14935060e-04],
 [ 1.65689310e+04,  3.97866901e+03, -4.26875993e+03,
 6.76414546e-01,  4.13886345e+00, -9.06248592e-04],
 [-6.35374702e+03,  3.75577856e+03,  9.25512406e+02,
 -2.46223398e+00, -9.41797616e-01,  2.32116636e-04],
 [ 6.19453762e+03,  9.98665444e+03, -2.04929624e+03,
 -5.29473684e+00,  2.58173149e+00, -7.99620325e-04],
 [-6.22493361e+03,  4.14255213e+03,  1.70132609e+02,
 -4.56347908e+00, -2.91130386e-01,  1.16287183e-04],
 [ 1.26414178e+04,  4.77187959e+03, -1.24228837e+03,
 -4.61458552e+00,  2.69335661e+00, -1.45531631e-03],
 [ 9.84362760e+03, -7.27541994e+01, -6.28943420e+02,
 1.00300985e+00,  1.38480213e+00, -7.40633352e-04],
 [ 2.98188991e+03, -1.00893870e+04,  2.27808565e+02,
 1.14142277e+01, -9.75699373e-01,  8.05879587e-04],
```

```
[ 1.41172177e+04, -3.44243277e+03, -2.22992877e+03,
  3.27253857e+00,  2.54207931e+00, -6.96688924e-04],
[-4.14234962e+03, -2.27164610e+04,  2.11534602e+03,
  1.33405330e+01, -3.51358258e+00,  1.34182072e-03],
[ 1.08440203e+04,  7.50164645e+02, -1.57311108e+03,
  8.71738784e-01,  2.16228530e+00, -6.99023279e-04],
[ 4.05808287e+03, -1.65960971e+04,  1.46543883e+01,
  1.16697659e+01, -1.07260412e+00,  7.30650437e-04],
[-7.01238772e+03,  5.60179493e+03,  1.11092455e+03,
 -3.87901714e+00, -8.87170417e-01,  1.29213577e-04],
[ 1.12920964e+04,  1.12855134e+04, -2.71874566e+03,
 -4.84866225e+00,  3.62942594e+00, -1.13298496e-03],
[ 5.11397115e+03,  6.31299340e+03, -1.06134401e+03,
 -3.37966123e+00,  1.73028694e+00, -6.51427979e-04],
[ 1.70639322e+04, -9.45908081e+03, -2.29701240e+03,
  8.07563256e+00,  2.33662892e+00, -4.94826467e-04],
[-5.89447275e+03,  2.57116029e+03,  1.17536884e+03,
 -2.62705553e+00, -1.00168340e+00,  1.33102988e-04],
[ 6.26937694e+03,  1.05364378e+04, -1.04786385e+03,
 -4.38333962e+00,  2.23855597e+00, -8.35217280e-04],
[ 8.52197882e+03,  5.82278047e+03, -1.70998120e+03,
 -8.92724248e-01,  2.33354476e+00, -6.20136500e-04],
[ 1.12358104e+04, -1.53238344e+04, -2.98060318e+03,
  1.00424477e+01,  1.15679670e+00,  2.91618637e-04],
[-2.01590332e+03,  9.18420633e+03,  1.47192929e+03,
 -4.23308319e+00,  2.21755181e-01, -3.62690851e-04],
[ 1.78418136e+04,  7.71978152e+03, -4.55783025e+03,
  2.32933996e+00,  4.76172829e+00, -7.64380722e-04],
[ 1.34980038e+03,  8.62610733e+03,  3.03391691e+02,
 -3.67052256e+00,  9.55817882e-01, -4.88982839e-04],
[ 1.74099348e+04, -8.34341460e+01, -4.17979248e+03,
  5.05001068e+00,  3.92158992e+00, -5.59289905e-04],
[ 7.89245794e+02,  6.32685108e+03,  7.04052839e+02,
 -3.59699061e+00,  4.20791966e-01, -4.97938680e-04],
[ 1.53677408e+04,  2.83919007e+03, -2.32897679e+03,
  4.52307764e+00,  3.46600195e+00, -6.11395503e-04],
[-5.40837671e+03,  5.18123798e+03,  9.30782498e+02,
 -3.90276204e+00, -6.13861745e-01,  5.82657830e-06],
[ 1.19947941e+04,  8.97876546e+03, -2.23653354e+03,
 -3.04780650e+00,  3.33807233e+00, -1.08674811e-03],
[ 8.63773613e+03,  5.68434255e+03, -9.59182210e+02,
  1.27095730e-01,  2.21240814e+00, -6.21301821e-04],
[ 8.21478900e+03, -1.25107205e+04, -3.20432697e+03,
  1.03854374e+01,  1.22111812e+00,  6.67285461e-04],
[ 2.83703514e+03,  1.14357633e+04, -2.41877599e+02,
 -3.46292083e+00,  1.62693424e+00, -5.17049848e-04],
[ 1.97595453e+04, -4.08393793e+03, -6.54629972e+03,
  6.50447975e+00,  4.27049394e+00, -3.68546476e-04],
[-2.59406187e+03,  9.54470078e+03,  5.10701711e+02,
 -4.30491629e+00,  2.92324806e-01, -2.20477961e-04],
[ 1.78036768e+04,  4.68114628e+03, -5.10693374e+03,
  7.33116013e-01,  4.54181200e+00, -9.04880818e-04]])
```

Stage 3.2: Solve for $null(\mathbf{L})$ and find ω

Solve the homogeneous system with SVD and take the last right-singular vector as $\mathbf{b}_\omega$.

Then form

$$\boldsymbol{\omega} = \begin{bmatrix} \omega_{11} & \omega_{12} & \omega_{13} \\ \omega_{12} & \omega_{22} & \omega_{23} \\ \omega_{13} & \omega_{23} & \omega_{33} \end{bmatrix}.$$

where $\mathbf{b}_\omega = [\omega_{11}, \omega_{12}, \omega_{22}, \omega_{13}, \omega_{23}, \omega_{33}]^\top$

The function `construct_bw_and_omega_from_L` below operationalises the same. The matrix derived is the direct input to Stage 4.

```
In [13]: def construct_bw_and_omega_from_L(L, tol=1e-4):
    _, _, Vt = np.linalg.svd(L)
    b_omega = Vt[-1]

    # Verify null-space solution using residual norm, not signed mean.
    residual = np.linalg.norm(L @ b_omega)
    if residual > tol:
        raise ValueError(f"Null-space residual too high: {residual}")

    omega = np.array(
        [
            [b_omega[0], b_omega[1], b_omega[3]],
            [b_omega[1], b_omega[2], b_omega[4]],
            [b_omega[3], b_omega[4], b_omega[5]],
        ],
        dtype=np.float64,
    )

    # Scale/sign normalise omega for stability.
    if not np.isclose(omega[2, 2], 0.0):
        omega = omega / omega[2, 2]
    if omega[0, 0] < 0:
        omega = -omega
        b_omega = -b_omega

    return b_omega, omega

b_omega, omega = construct_bw_and_omega_from_L(L)
```

Stage 3.3: Deliverable

For a valid camera, $\boldsymbol{\omega} = (\mathbf{K}\mathbf{K}^\top)^{-1} \implies \boldsymbol{\omega} = \mathbf{K}^{-\top}\mathbf{K}^{-1}$.

Now, for any non-zero vector $\mathbf{x}$,

$$\mathbf{x}^\top \boldsymbol{\omega} \mathbf{x} = (\mathbf{K}^{-1}\mathbf{x})^\top (\mathbf{K}^{-1}\mathbf{x}) = \|\mathbf{K}^{-1}\mathbf{x}\|^2 > 0,$$

so $\boldsymbol{\omega}$ must always be positive definite for a real camera.

This is also confirmed by the code below.

```
In [14]: print("b_omega:")
    print(b_omega)

    print("\nomega:")
    print(omega)
```

```

omega_eigs = np.linalg.eigvals(omega)
print("\nomega eigenvalues:", omega_eigs)
print("omega is positive definite:", bool(np.all(np.real(omega_eigs) > 0))

print("\nChecking linear constraints via v^T b (should be ~0):")
_ = construct_L_matrix(H_i_all, b_omega=b_omega, print_vtb=True)

```

b_omega:

```

[-1.01585174e-07  1.78947625e-10 -1.01205515e-07  1.04809447e-04
 6.88011865e-05 -9.99999992e-01]

```

omega:

```

[[ 1.01585175e-07 -1.78947627e-10 -1.04809448e-04]
 [-1.78947627e-10  1.01205515e-07 -6.88011871e-05]
 [-1.04809448e-04 -6.88011871e-05  1.00000000e+00]]

```

omega eigenvalues: [1.00000002e+00 8.55842447e-08 1.01487821e-07]

omega is positive definite: True

Checking linear constraints via v^T b (should be ~0):

Row-wise check (v^T b):

```

[-7.97203565e-06 -4.28728461e-06  1.75786035e-07  3.34570376e-06
 1.02807797e-06 -6.82962916e-06 -3.40004648e-06 -1.07473428e-05
 4.31470162e-06  1.42854568e-05 -1.30639065e-05 -2.60492748e-05
 -6.51530024e-06 -9.44862688e-06  2.06085748e-06  6.60437989e-06
 -3.73837488e-07 -5.00270181e-06  8.39710776e-06  2.06988838e-06
 1.58003068e-06 -1.45847106e-05  2.45719531e-06  1.14782084e-05
 -2.52718255e-06  2.22204933e-06  1.27089081e-06 -6.30013925e-07
 4.70717968e-06 -4.46671911e-06  5.54500048e-06  1.73060930e-05
 -3.09240772e-06  1.96085317e-06  4.11577119e-06  4.57013984e-06
 5.29325316e-06 -6.72998378e-07  2.93710630e-06  8.76496929e-07
 -4.48399693e-06 -1.96670264e-06 -8.25721975e-06 -1.36746493e-05
 3.75780139e-06  1.28006261e-05 -4.06460987e-07 -9.99456253e-07
 -9.69586347e-07  6.43552810e-06  7.46475065e-06 -7.22310679e-06
 4.36307190e-06 -1.39358754e-06  2.93475512e-06  3.29827220e-06]

```

max |v^T b|: 2.6049274780860603e-05

Stage 4: Recover Intrinsic Matrix $\mathbf{K}$

Stage 4.1: Intrinsic Recovery (Analytic + Cholesky Verification)

Following Zhang's original paper, write

$$\boldsymbol{\omega} = \begin{bmatrix} \omega_{11} & \omega_{12} & \omega_{13} \\ \omega_{12} & \omega_{22} & \omega_{23} \\ \omega_{13} & \omega_{23} & \omega_{33} \end{bmatrix} = (\mathbf{K}\mathbf{K}^\top)^{-1}.$$

Let

$$\mathbf{K} = \begin{bmatrix} \alpha_x & s & x_0 \\ 0 & \alpha_y & y_0 \\ 0 & 0 & 1 \end{bmatrix},$$

We implement both routes below:

- Appendix B closed-form recovery (analytic),
- Cholesky recovery from $\boldsymbol{\omega} = \mathbf{K}^{-\top} \mathbf{K}^{-1}$,

and verify they are numerically equivalent in the same cell. The downstream default is set to the Cholesky version so later function calls are unchanged.

For the Cholesky route, we use the fact that a real camera implies ω is symmetric positive definite (SPD), so it admits

$$\omega = \mathbf{L}\mathbf{L}^\top,$$

with $\mathbf{L}$ lower triangular. Comparing with

$$\omega = \mathbf{K}^{-\top}\mathbf{K}^{-1},$$

we identify $\mathbf{L} = \mathbf{K}^{-\top}$, hence

$$\mathbf{K} = \left(\mathbf{L}^{-1}\right)^\top.$$

The closed-form expressions are:

$$y_0 = \frac{\omega_{12}\omega_{13} - \omega_{11}\omega_{23}}{\omega_{11}\omega_{22} - \omega_{12}^2},$$

$$\lambda = \omega_{33} - \frac{\omega_{13}^2 + y_0(\omega_{12}\omega_{13} - \omega_{11}\omega_{23})}{\omega_{11}},$$

$$\alpha_x = \sqrt{\frac{\lambda}{\omega_{11}}}, \quad \alpha_y = \sqrt{\frac{\lambda\omega_{11}}{\omega_{11}\omega_{22} - \omega_{12}^2}},$$

$$s = -\frac{\omega_{12}\alpha_x^2\alpha_y}{\lambda},$$

$$x_0 = \frac{sy_0}{\alpha_y} - \frac{\omega_{13}\alpha_x^2}{\lambda}.$$

```
In [15]: def _normalise_intrinsics_matrix(K):
    K = K.astype(np.float64)
    if not np.isclose(K[2, 2], 0.0):
        K = K / K[2, 2]

    # Enforce positive diagonal without changing K K^T.
    signs = np.sign(np.diag(K))
    signs[signs == 0] = 1.0
    D = np.diag(signs)
    K = K @ D

    if not np.isclose(K[2, 2], 0.0):
        K = K / K[2, 2]
    return K

def recover_intrinsics_from_omega_analytic(omega):
    y_0 = (omega[0, 1] * omega[0, 2] - omega[0, 0] * omega[1, 2]) / (
        omega[0, 0] * omega[1, 1] - omega[0, 1] ** 2
    )
    lambda_param = (
```

```

        omega[2, 2]
        - (
            omega[0, 2] ** 2
            + y_0 * (omega[0, 1] * omega[0, 2] - omega[0, 0] * omega[1, 2]
        )
        / omega[0, 0]
    )
    alpha_x = (lambda_param / omega[0, 0]) ** 0.5
    alpha_y = (
        (lambda_param * omega[0, 0]) / (omega[0, 0] * omega[1, 1] - omega
    ) ** 0.5
    s = -(omega[0, 1] * (alpha_x**2) * alpha_y) / lambda_param
    x_0 = ((s * y_0) / alpha_y) - (omega[0, 2] * (alpha_x**2) / lambda_pa

    K = np.array([[alpha_x, s, x_0], [0, alpha_y, y_0], [0, 0, 1]], dtype
    return _normalise_intrinsic_matrix(K)

def recover_intrinsic_matrix_from_omega_cholesky(omega):
    omega_sym = 0.5 * (omega + omega.T)
    L = np.linalg.cholesky(omega_sym) # omega = L L^T, L lower triangular
    K = np.linalg.inv(L).T # since L = K^{-T}
    return _normalise_intrinsic_matrix(K)

# Verify equivalence of analytic and Cholesky recoveries on the current o
K_from_analytic = recover_intrinsic_matrix_from_omega_analytic(omega)
K_from_cholesky = recover_intrinsic_matrix_from_omega_cholesky(omega)
K_match = np.allclose(K_from_analytic, K_from_cholesky, rtol=1e-5, atol=1

print("K (analytic) =\n", K_from_analytic)
print("\nK (cholesky) =\n", K_from_cholesky)
print("\nmax |delta| =", float(np.max(np.abs(K_from_analytic - K_from_cho
print("Equivalent (allclose):", bool(K_match))

# Keep downstream API unchanged.
recover_intrinsic_matrix_from_omega = recover_intrinsic_matrix_from_omega_cholesky

```

```

K (analytic) =
[[2.88384643e+03 5.08957469e+00 1.03294035e+03]
 [0.00000000e+00 2.88925506e+03 6.81642982e+02]
 [0.00000000e+00 0.00000000e+00 1.00000000e+00]]

```

```

K (cholesky) =
[[2.88384643e+03 5.08957469e+00 1.03294035e+03]
 [3.92782980e-13 2.88925506e+03 6.81642982e+02]
 [6.20643563e-16 6.1112339e-19 1.00000000e+00]]

```

```

max |delta| = 9.094947017729282e-13
Equivalent (allclose): True

```

Stage 4.2: Recover Intrinsic Matrix

```

In [16]: K = recover_intrinsic_matrix_from_omega(omega)
print("Intrinsic Matrix K = \n", K)

```

Intrinsic Matrix $K =$

```
[[2.88384643e+03 5.08957469e+00 1.03294035e+03]
 [3.92782980e-13 2.88925506e+03 6.81642982e+02]
 [6.20643563e-16 6.11112339e-19 1.00000000e+00]]
```

Thus, $\alpha_x \approx 2883.84$, $\alpha_y \approx 2889.26$, $s \approx 5.09$, $x_0 \approx 1032.94$, $y_0 \approx 681.64$.

Since $\frac{|s|}{\alpha_x} \approx 0.18\%$, the skew is very close to zero, which is consistent with near-orthogonal pixel axes in the image sensor. We can note that this ratio is useful because it normalises skew (in px) by focal-scale magnitude (also in px), giving us a way to judge whether skew is present that is independent of units. We will see that in the OpenCV calibration, s is actually 0.

Stage 4.3: Checking agreement between α_x, α_y

The code computes

$$\text{alpha_xy_error_wrt_x} = 100 \cdot \frac{K_{11} - K_{22}}{K_{11}} = 100 \cdot \frac{\alpha_x - \alpha_y}{\alpha_x}.$$

Interpretation:

- This is the signed percentage difference between α_x and α_y , normalised by α_x .
- A value close to 0 means near-square pixel scaling in the two image axes.
- Positive value means $\alpha_x > \alpha_y$; negative means $\alpha_x < \alpha_y$.

```
In [17]: # alpha_y within 1% of alpha_x
alpha_xy_error_wrt_x = ((K[0, 0] - K[1, 1]) / K[0, 0]) * 100
print(alpha_xy_error_wrt_x, "% relative difference")
```

-0.18754926840000047 % relative difference

Stage 4.4 Compare with `cv.calibrateCamera`

Use OpenCV as a black-box reference on the same corner detections and compare

$$(\alpha_x, \alpha_y, x_0, y_0, s)$$

between custom and OpenCV intrinsics.

```
In [18]: # Compare custom K against cv.calibrateCamera (reusing precomputed corner
image_names_cv = list(images_list_filtered)
imgpoints_cv = [
    stage1_corners_by_image[name].astype(np.float32) for name in image_na
]

# Reuse Stage 2 world grid and set z=0 for OpenCV object points.
objp_cv = np.column_stack(
    [
        X_world_h[:, 0],
        X_world_h[:, 1],
        np.zeros(X_world_h.shape[0]),
    ]
).astype(np.float32)
objpoints_cv = [objp_cv.copy() for _ in image_names_cv]
```

```

sample_img = cv.imread(os.path.join(root, image_names_cv[0]))
image_size = (sample_img.shape[1], sample_img.shape[0])

rms_cv, K_opencv, dist_opencv, _, _ = cv.calibrateCamera(
    objpoints_cv, imgpoints_cv, image_size, None, None
)

# Store in dataframe
df_K_compare = pd.DataFrame(
    [
        ("alpha_x", K[0, 0], K_opencv[0, 0]),
        ("alpha_y", K[1, 1], K_opencv[1, 1]),
        ("x_0", K[0, 2], K_opencv[0, 2]),
        ("y_0", K[1, 2], K_opencv[1, 2]),
        ("s", K[0, 1], K_opencv[0, 1]),
    ],
    columns=["Parameter", "Custom K", "OpenCV K"],
)

df_K_compare["Abs Diff"] = (df_K_compare["Custom K"] - df_K_compare["Open
df_K_compare["Rel Diff (%)"] = np.where(
    np.isclose(df_K_compare["OpenCV K"], 0.0),
    np.nan,
    100.0 * df_K_compare["Abs Diff"] / np.abs(df_K_compare["OpenCV K"]),
)

print(f"OpenCV RMS reprojection error: {rms_cv:.4f} px")
print("OpenCV distortion coefficients:\n", dist_opencv.ravel())
display(df_K_compare)

```

OpenCV RMS reprojection error: 0.4438 px

OpenCV distortion coefficients:

```

[-4.23038540e-02 -7.49437613e-02 -2.49848508e-03  2.14488607e-03
 2.33536319e+00]

```

	Parameter	Custom K	OpenCV K	Abs Diff	Rel Diff (%)
0	alpha_x	2883.846431	2882.246805	1.599626	0.055499
1	alpha_y	2889.255064	2906.705466	17.450402	0.600350
2	x_0	1032.940350	1046.745185	13.804835	1.318834
3	y_0	681.642982	693.286481	11.643498	1.679464
4	s	5.089575	0.000000	5.089575	NaN

Stage 4.5: Deliverable Reporting

```

In [19]: stage4_intrinsics = pd.DataFrame(
    {
        "parameter": ["alpha_x", "alpha_y", "skew_s", "x_0", "y_0"],
        "value": [K[0, 0], K[1, 1], K[0, 1], K[0, 2], K[1, 2]],
    }
)

print("Recovered K:\n", K)
print("\nRelative alpha mismatch wrt x (%):", float(alpha_xy_error_wrt_x))
print("Absolute skew magnitude:", float(abs(K[0, 1])))

```

```
print("\nCustom vs OpenCV comparison table:")

# display(stage4_intrinsics)
display(df_K_compare)
```

Recovered K:

```
[[2.88384643e+03 5.08957469e+00 1.03294035e+03]
 [3.92782980e-13 2.88925506e+03 6.81642982e+02]
 [6.20643563e-16 6.11112339e-19 1.00000000e+00]]
```

Relative alpha mismatch wrt x (%): -0.18754926840000047

Absolute skew magnitude: 5.0895746876690025

Custom vs OpenCV comparison table:

	Parameter	Custom K	OpenCV K	Abs Diff	Rel Diff (%)
0	alpha_x	2883.846431	2882.246805	1.599626	0.055499
1	alpha_y	2889.255064	2906.705466	17.450402	0.600350
2	x_0	1032.940350	1046.745185	13.804835	1.318834
3	y_0	681.642982	693.286481	11.643498	1.679464
4	s	5.089575	0.000000	5.089575	NaN

Stage 5: Extrinsic Recovery and Reprojection

Stage 5.1: Recover Extrinsics and Evaluate Full Reprojection

For each view i , with

$$\mathbf{H}_i = [\mathbf{h}_{1i} \ \mathbf{h}_{2i} \ \mathbf{h}_{3i}], \quad \mathbf{H}_i = \mathbf{K}[\mathbf{r}_{1i} \ \mathbf{r}_{2i} \ \mathbf{t}_i],$$

`get_view_extrinsics_from_intrinsics` implements:

1. Invert intrinsics:

$$\mathbf{K}^{-1}\mathbf{h}_{1i}, \ \mathbf{K}^{-1}\mathbf{h}_{2i}, \ \mathbf{K}^{-1}\mathbf{h}_{3i}.$$

2. Compute two scale estimates from the first two columns:

$$m_{1i} = \frac{1}{\|\mathbf{K}^{-1}\mathbf{h}_{1i}\|_2}, \quad m_{2i} = \frac{1}{\|\mathbf{K}^{-1}\mathbf{h}_{2i}\|_2},$$

and average them:

$$m_i = \frac{m_{1i} + m_{2i}}{2}.$$

3. Form scaled columns:

$$\hat{\mathbf{r}}_{1i} = m_i \mathbf{K}^{-1}\mathbf{h}_{1i}, \quad \hat{\mathbf{r}}_{2i} = m_i \mathbf{K}^{-1}\mathbf{h}_{2i}, \quad \hat{\mathbf{t}}_i = m_i \mathbf{K}^{-1}\mathbf{h}_{3i},$$

$$\hat{\mathbf{r}}_{3i} = \hat{\mathbf{r}}_{1i} \times \hat{\mathbf{r}}_{2i}.$$

4. Build

$$\mathbf{Q}_i = [\hat{\mathbf{r}}_{1i} \ \hat{\mathbf{r}}_{2i} \ \hat{\mathbf{r}}_{3i}],$$

and project to the nearest valid rotation by SVD:

$$\mathbf{Q}_i = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^\top, \quad \mathbf{R}_i = \mathbf{U}\mathbf{V}^\top.$$

If $\det(\mathbf{R}_i) < 0$, flip sign to keep a proper orientation.

The returned pair is $(\mathbf{R}_i, \mathbf{t}_i)$, then used in

$$\mathbf{P}_i = \mathbf{K}[\mathbf{R}_i \mid \mathbf{t}_i]$$

for reprojection error computation.

```
In [20]: # Extrinsic for H_1
def get_view_extrinsics_from_intrinsics(K, H):

    K_inv = np.linalg.inv(K)

    h1, h2, h3 = H[:, 0], H[:, 1], H[:, 2]

    m1 = 1 / np.linalg.norm((K_inv @ h1))
    m2 = 1 / np.linalg.norm((K_inv @ h2))

    m = 0.5 * (m1 + m2)

    r1_hat = m * (K_inv @ h1)
    r2_hat = m * (K_inv @ h2)
    r3_hat = np.cross(r1_hat, r2_hat)

    t = m * (K_inv @ h3)

    Q = np.column_stack([r1_hat, r2_hat, r3_hat])
    U, _, Vt = np.linalg.svd(Q)
    R = U @ Vt

    if np.linalg.det(R) < 0:
        R = -R
        t = -t

    return R, t

R, t = get_view_extrinsics_from_intrinsics(K, H_i_all[0])
R, t
```

```
Out[20]: (array([[ 0.3046152 ,  0.95080125, -0.05644961],
                 [-0.3593347 ,  0.16960511,  0.91766698],
                 [ 0.88209305, -0.259251 ,  0.39332017]]),
          array([-4.48736756,  0.18852052, 18.69713406]))
```

Stage 5.2: View-wise Reprojection Error from $\mathbf{P}_i$

For each retained view i , the code evaluates reprojection quality using the camera matrix

$$\mathbf{P}_i = \mathbf{K}[\mathbf{R}_i \mid \mathbf{t}_i].$$

The checkerboard points are represented in homogeneous world coordinates as

$$\mathbf{X}_j = [X_j \ Y_j \ 0 \ 1]^\top, \quad j = 1, \dots, N,$$

where

$$N = n_{\text{rows}} n_{\text{cols}}, \quad (n_{\text{cols}}, n_{\text{rows}}) = \text{CHESSBOARD_SIZE}.$$

For each view i :

1. Recover extrinsics from homography $\mathbf{H}_i$:

$$(\mathbf{R}_i, \mathbf{t}_i) = \text{get_view_extrinsics_from_intrinsics}(\mathbf{K}, \mathbf{H}_i).$$

2. Build the projection matrix:

$$\mathbf{P}_i = \text{get_ProjectionMatrix_for_view}(\mathbf{K}, \mathbf{R}_i, \mathbf{t}_i).$$

3. Project each world point and dehomogenize:

$$\hat{\mathbf{x}}_{ij}^{(h)} = \mathbf{P}_i \mathbf{X}_j = [\hat{x}_{ij} \ \hat{y}_{ij} \ \hat{w}_{ij}]^\top,$$

$$\hat{\mathbf{x}}_{ij} = \begin{bmatrix} \hat{x}_{ij}/\hat{w}_{ij} \\ \hat{y}_{ij}/\hat{w}_{ij} \end{bmatrix}.$$

4. Compare to detected corner $\tilde{\mathbf{x}}_{ij}$ from `get_corners(...)`:

$$e_{ij} = \|\hat{\mathbf{x}}_{ij} - \tilde{\mathbf{x}}_{ij}\|_2.$$

5. Aggregate per view:

$$\bar{e}_i = \frac{1}{N} \sum_{j=1}^N e_{ij}, \quad e_i^{\max} = \max_{1 \leq j \leq N} e_{ij}.$$

In code:

- $\bar{e}_i \leftrightarrow \text{Mean_reprojection_error_fromP[image_name]}$
- $e_i^{\max} \leftrightarrow \text{Max_reprojection_error_fromP[image_name]}$

These are collected into `df_rep_error_fromP_perView`

```
In [21]: # View-wise reprojection error from P (mean + max per view)
get_ProjectionMatrix_for_view = lambda K, R, t: K @ np.column_stack([R, t])

n_cols, n_rows = CHESSBOARD_SIZE
world_points = np.array(
    [(x, y, 0.0, 1.0) for y in range(n_rows) for x in range(n_cols)], dtype=float
)

Mean_reprojection_error_fromP = {}
Max_reprojection_error_fromP = {}

if len(H_i_all) != len(images_list_filtered):
    raise ValueError(
        f"Mismatch: homographies={len(H_i_all)}, images={len(images_list_filtered)}"
    )
```

```

for image_name, H_i in zip(images_list_filtered, H_i_all):
    R_v, t_v = get_view_extrinsics_from_intrinsics(K, H_i)
    P_v = get_ProjectionMatrix_for_view(K, R_v, t_v)

    corners_v = get_corners(root + image_name).reshape(-1, 2).astype(np.float32)

    per_corner_error = []
    for corner_idx in range(len(world_points)):
        X = world_points[corner_idx]
        xhat_h = P_v @ X
        xhat = xhat_h[:2] / xhat_h[2]
        err = np.linalg.norm(xhat - corners_v[corner_idx])
        per_corner_error.append(err)

    Mean_reprojection_error_fromP[image_name] = float(np.mean(per_corner_error))
    Max_reprojection_error_fromP[image_name] = float(np.max(per_corner_error))

```

Stage 5.3: Deliverables

```

In [22]: dict_stage5 = {
    "Image": list(Mean_reprojection_error_fromP.keys()),
    "Mean Reprojection Error from P (px)": list(Mean_reprojection_error_fromP.values()),
    "Max Reprojection Error from P (px)": [
        Max_reprojection_error_fromP[k] for k in Mean_reprojection_error_fromP.keys()
    ],
}

df_rep_error_fromP_perView = pd.DataFrame(dict_stage5)
display(df_rep_error_fromP_perView)

plot_cols = [
    "Mean Reprojection Error from P (px)",
    "Max Reprojection Error from P (px)",
]
df_plot = df_rep_error_fromP_perView.set_index("Image")

# Show mean and max errors side by side for each image.
ax = df_plot[plot_cols].plot(
    kind="bar", figsize=(16, 5), width=0.8, color=[COLOUR_NAVY, COLOUR_TEAL]
)
ax.set_title("Reprojection Error per Image (Grouped)")
ax.set_ylabel("Error (px)")
ax.set_xlabel("Image")
ax.tick_params(axis="x", rotation=75)
plt.tight_layout()
save_plot("Stage 5", "Per-view grouped reprojection errors")
plt.show()

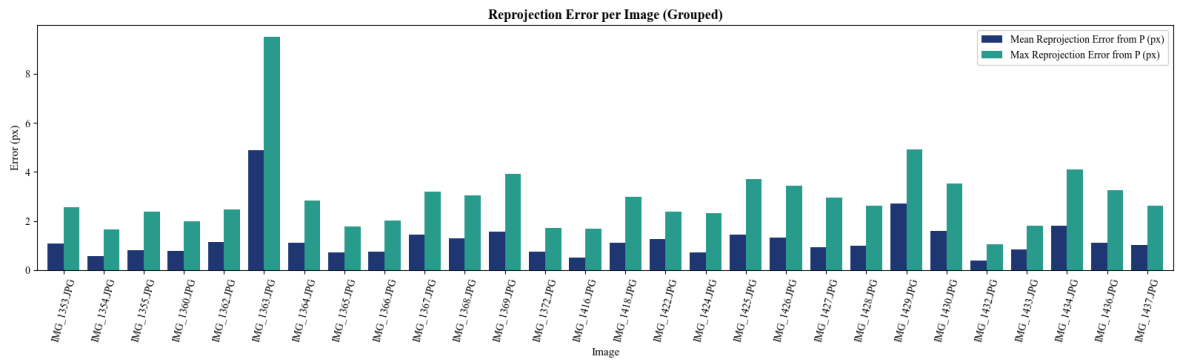
# Show the same quantities as trends across image index.
plt.figure(figsize=(16, 5))
plt.plot(
    df_plot.index, df_plot[plot_cols[0]], marker="o", label="Mean", color=COLOUR_NAVY
)
plt.plot(
    df_plot.index, df_plot[plot_cols[1]], marker="s", label="Max", color=COLOUR_TEAL
)
plt.title("Reprojection Error Trend Across Images")
plt.ylabel("Error (px)")
plt.xlabel("Image")

```

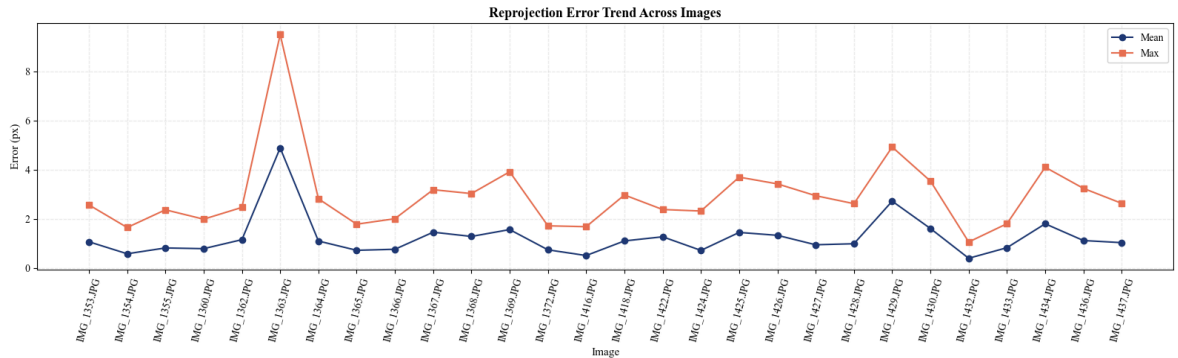
```
plt.xticks(rotation=75)
plt.legend()
plt.grid(alpha=0.25)
plt.tight_layout()
save_plot("Stage 5", "Per-view reprojection error trends")
plt.show()
```

	Image	Mean Reprojection Error from P (px)	Max Reprojection Error from P (px)
0	IMG_1353.JPG	1.065142	2.567323
1	IMG_1354.JPG	0.576987	1.646116
2	IMG_1355.JPG	0.814079	2.367370
3	IMG_1360.JPG	0.785277	1.985715
4	IMG_1362.JPG	1.151269	2.465152
5	IMG_1363.JPG	4.888183	9.505435
6	IMG_1364.JPG	1.095746	2.816993
7	IMG_1365.JPG	0.718515	1.781374
8	IMG_1366.JPG	0.760243	2.005135
9	IMG_1367.JPG	1.455826	3.186517
10	IMG_1368.JPG	1.282866	3.028076
11	IMG_1369.JPG	1.562134	3.916137
12	IMG_1372.JPG	0.737367	1.714647
13	IMG_1416.JPG	0.505287	1.679802
14	IMG_1418.JPG	1.100093	2.970957
15	IMG_1422.JPG	1.268420	2.378616
16	IMG_1424.JPG	0.714163	2.318131
17	IMG_1425.JPG	1.446040	3.697933
18	IMG_1426.JPG	1.328130	3.423069
19	IMG_1427.JPG	0.942948	2.941723
20	IMG_1428.JPG	0.985846	2.617722
21	IMG_1429.JPG	2.718407	4.924307
22	IMG_1430.JPG	1.605244	3.538279
23	IMG_1432.JPG	0.393710	1.056922
24	IMG_1433.JPG	0.823927	1.800886
25	IMG_1434.JPG	1.797823	4.101362
26	IMG_1436.JPG	1.115848	3.239954
27	IMG_1437.JPG	1.029812	2.630720

Saved plot: /Users/tejasrao/Desktop/Master/Studies_at_Krea/Year 3/T9/CV/C
alibrationProject/plots/stage_5__per_view_grouped_reprojection_errors.png



Saved plot: /Users/tejasrao/Desktop/Master/Studies_at_Krea/Year 3/T9/CV/C alibrationProject/plots/stage_5__per_view_reprojection_error_trends.png



Plot Interpretation

The grouped bars and trend lines separate per-view mean and worst-case reprojection errors from $\mathbf{P}_i$. Large gaps between mean and max for a view indicate local outliers or uneven geometric fit, while flatter trends indicate more uniform reprojection behaviour across views.

Stage 6: Sensitivity to Number of Views

This section will rerun calibration with varying number of views $n = 3, 4, 5, \dots 28$. Once we have this, we plot the curve: $\bar{e}(n)$ vs n where $\bar{e}(n)$ is the mean reprojection error for that subset size.

Stage 6.1: Functionalising above pipeline

Rewrite calibration schema described in Stages 1-5 into a single modular function.

```
In [23]: # Full pipeline
# Cached per-image intermediates to speed repeated Stage 6 subset runs.
def run_full_calibration(images_list, dataset_root=root):
    def to_path(img_name):
        if os.path.isfile(img_name):
            return img_name
        if dataset_root is None:
            return img_name
        return os.path.join(dataset_root, os.path.basename(img_name))

    images_list_filtered = list(images_list)

    cache = globals().setdefault(
        "_calib_cache",
```

```

        {"H": {}, "mean_e": {}, "corners": {}, "world_points": None},
    )

    if cache["world_points"] is None:
        n_cols, n_rows = CHESSBOARD_SIZE
        cache["world_points"] = np.array(
            [(x, y, 0.0, 1.0) for y in range(n_rows) for x in range(n_cols)],
            dtype=np.float64,
        )
    world_points = cache["world_points"]

    # Homography + homography reprojection error (normalised DLT), cached
    H_i_all = []
    mean_e_i_all = []
    for img_name in images_list_filtered:
        if img_name not in cache["H"]:
            H_i, mean_e_i = Homography_definition_Normalised(to_path(img_name))
            cache["H"][img_name] = H_i
            cache["mean_e"][img_name] = float(mean_e_i)

        H_i_all.append(cache["H"][img_name])
        mean_e_i_all.append(cache["mean_e"][img_name])

    # Step 2/3/4: L --> omega --> K
    L = construct_L_matrix(H_i_all)
    b_omega, omega = construct_bw_and_omega_from_L(L)
    K = recover_intrinsics_from_omega(omega)

    # Step 5: Projection matrix + per-view mean/max reprojection error from P
    Mean_reprojection_error_fromP = {}
    Max_reprojection_error_fromP = {}

    stage1_cache = globals().get("stage1_corners_by_image", {})

    for image_name, H_i in zip(images_list_filtered, H_i_all):
        R_v, t_v = get_view_extrinsics_from_intrinsics(K, H_i)
        P_v = K @ np.column_stack([R_v, t_v])

        if image_name in stage1_cache:
            corners_v = stage1_cache[image_name].reshape(-1, 2).astype(np.float64)
        else:
            if image_name not in cache["corners"]:
                cache["corners"][image_name] = (
                    get_corners(to_path(image_name)).reshape(-1, 2).astype(np.float64)
                )
            corners_v = cache["corners"][image_name]

        # Vectorized reprojection error over all checkerboard points.
        xhat_h = (P_v @ world_points.T).T
        xhat = xhat_h[:, :2] / xhat_h[:, [2]]
        per_corner_error = np.linalg.norm(xhat - corners_v, axis=1)

        Mean_reprojection_error_fromP[image_name] = float(per_corner_error.mean())
        Max_reprojection_error_fromP[image_name] = float(per_corner_error.max())

    df_rep_error_fromP_perView = pd.DataFrame(
        {
            "Image": list(Mean_reprojection_error_fromP.keys()),
            "Mean Reprojection Error from P (px)": list(
                Mean_reprojection_error_fromP.values()
            ),
        }
    )

```



```

        out_seed["overall_mean_reproj_error_px"]
    ),
    "alpha_x": float(K_seed[0, 0]),
    "alpha_y": float(K_seed[1, 1]),
}
except Exception:
    # Keep the run moving if one subset is numerically unstable.
    subset_stats_cache[subset_key] = {
        "mean_reprojection_error_px": np.nan,
        "alpha_x": np.nan,
        "alpha_y": np.nan,
    }

# Record per-seed/per-n statistics for downstream aggregation.
stats = subset_stats_cache[subset_key]
seed_curve_rows.append(
    {
        "seed": seed,
        "num_views": num_views_now,
        "mean_reprojection_error_px": stats["mean_reprojection_er
        "alpha_x": stats["alpha_x"],
        "alpha_y": stats["alpha_y"],
    }
)

if len(working_list_seed) == 3:
    break

# Randomly remove one view to move from n to n-1.
remove_img = rng_seed.choice(working_list_seed)
working_list_seed.remove(remove_img)

df_stage6_seed_curves = pd.DataFrame(seed_curve_rows)

# Plot one trajectory per seed to see sampling variability.
fig, axes = plt.subplots(10, 10, figsize=(30, 30), sharex=True, sharey=True)
axes = axes.flatten()

for ax, seed in zip(axes, seed_values):
    df_seed = (
        df_stage6_seed_curves[df_stage6_seed_curves["seed"] == seed]
        .sort_values("num_views")
        .reset_index(drop=True)
    )
    ax.plot(
        df_seed["num_views"],
        df_seed["mean_reprojection_error_px"],
        marker="o",
        linewidth=1.3,
        color=COLOUR_NAVY,
    )
    ax.set_title(f"Seed {seed}", fontsize=10)
    ax.grid(alpha=0.25)

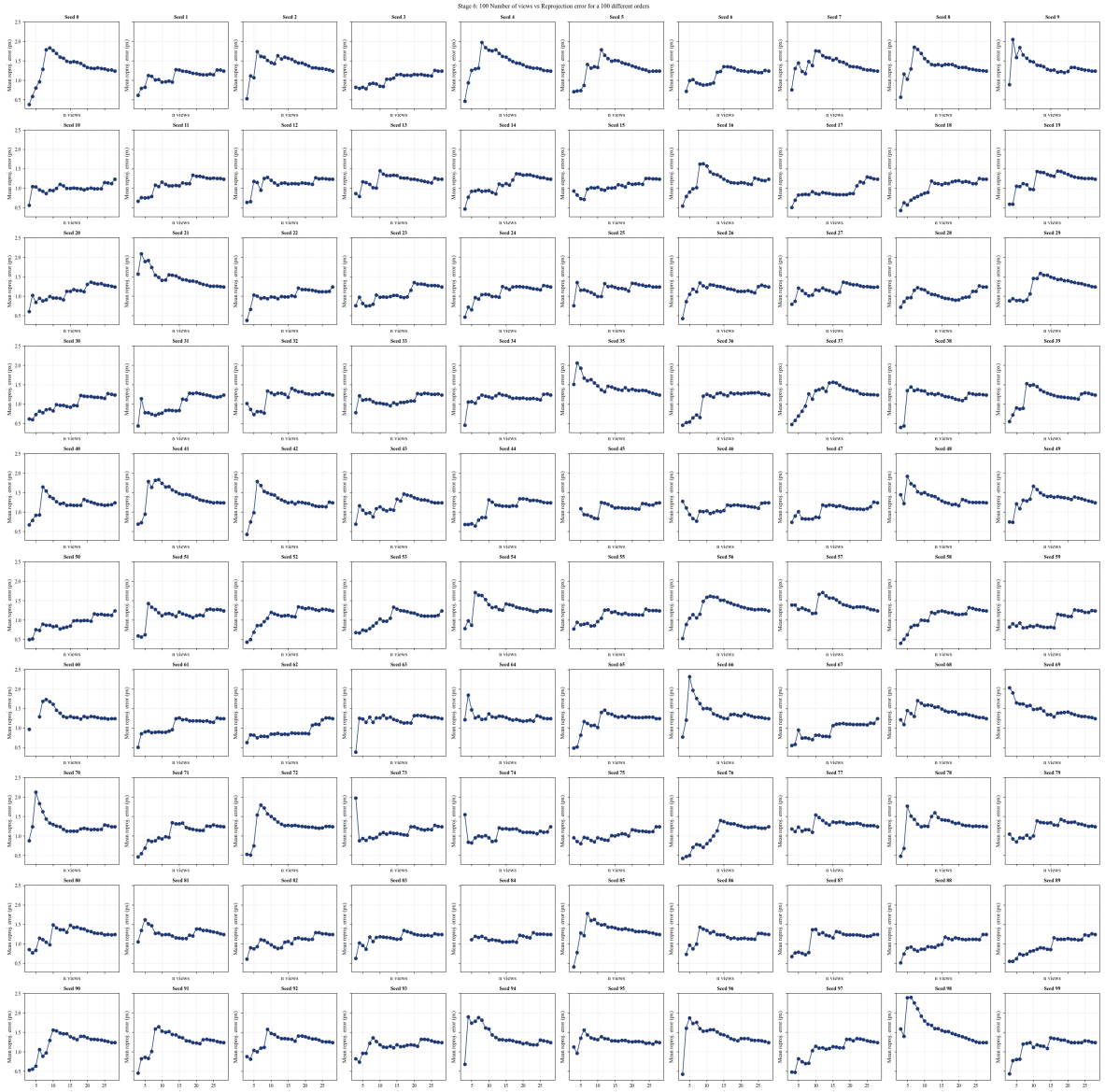
for ax in axes:
    ax.set_xlabel("n views")
    ax.set_ylabel("Mean reproj. error (px)")

plt.suptitle(

```

```
f"Stage 6: {num_seed_runs} Number of views vs Reprojection error for
y=0.995,
)
plt.tight_layout()
save_plot("Stage 6.2", "Random-seed trajectory grid", fig=fig)
plt.show()
```

Saved plot: /Users/tejasrao/Desktop/Master/Studies_at_Krea/Year 3/T9/CV/C
alibrationProject/plots/stage_6_2_random_seed_trajectory_grid.png



Stage 6.3: Mean Reprojection Error with SEM vs Number of Views (Deliverable)

This section reports how calibration error changes with the number of planes/views, and quantifies uncertainty across random subset choices.

The code below aggregates seed-wise mean reprojection error by n , computes standard error of the mean (SEM), and plots the deliverable curve used for the Stage 6 number-of-views analysis.

```
In [25]: # Stage 6.3 Deliverable Code Cell: Error bars across seeds for each n_vie
# Keep successful runs only, then aggregate by view count.
df_stage6_errorbar = (
    df_stage6_seed_curves.dropna(subset=["mean_reprojection_error_px"])
```

```

        .groupby("num_views")["mean_reprojection_error_px"]
        .agg(mean_error_px="mean", std_error_px="std", num_seed_curves="count")
        .reset_index()
        .sort_values("num_views")
        .reset_index(drop=True)
    )
    df_stage6_errorbar["sem_error_px"] = df_stage6_errorbar["std_error_px"] /
        df_stage6_errorbar["num_seed_curves"]
    )

    display(df_stage6_errorbar)

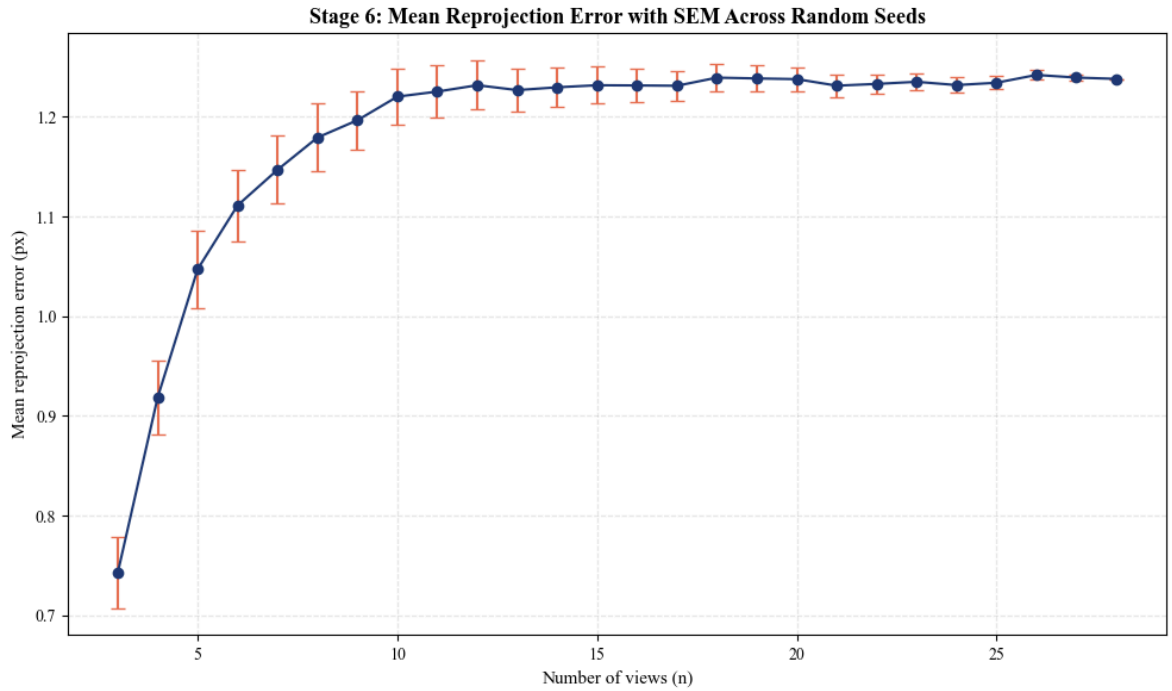
# Plot mean reprojection error with SEM bars against number of views.

plt.figure(figsize=(10, 6))
plt.errorbar(
    df_stage6_errorbar["num_views"],
    df_stage6_errorbar["mean_error_px"],
    yerr=df_stage6_errorbar["sem_error_px"],
    marker="o",
    capsize=4,
    color=COLOUR_NAVY,
    ecolor=COLOUR_CORAL,
)
plt.xlabel("Number of views (n)")
plt.ylabel("Mean reprojection error (px)")
plt.title("Stage 6: Mean Reprojection Error with SEM Across Random Seeds")
plt.grid(alpha=0.3)
plt.tight_layout()
save_plot("Stage 6.3", "Mean reprojection error with SEM")
plt.show()

```

	num_views	mean_error_px	std_error_px	num_seed_curves	sem_error_px
0	3	0.742803	0.347330	96	0.035449
1	4	0.918797	0.364578	97	0.037017
2	5	1.047221	0.385151	99	0.038709
3	6	1.111277	0.359568	100	0.035957
4	7	1.147010	0.342809	100	0.034281
5	8	1.179358	0.338181	100	0.033818
6	9	1.196697	0.293778	100	0.029378
7	10	1.220381	0.278353	100	0.027835
8	11	1.225435	0.261169	100	0.026117
9	12	1.231836	0.246059	100	0.024606
10	13	1.226948	0.218164	100	0.021816
11	14	1.229731	0.198759	100	0.019876
12	15	1.231813	0.186239	100	0.018624
13	16	1.231612	0.163186	100	0.016319
14	17	1.231286	0.151641	100	0.015164
15	18	1.239440	0.142530	100	0.014253
16	19	1.238632	0.131315	100	0.013131
17	20	1.237935	0.119987	100	0.011999
18	21	1.231373	0.113378	100	0.011338
19	22	1.233133	0.096160	100	0.009616
20	23	1.235252	0.086437	100	0.008644
21	24	1.231971	0.077119	100	0.007712
22	25	1.234317	0.063599	100	0.006360
23	26	1.242160	0.046698	100	0.004670
24	27	1.239480	0.033807	100	0.003381
25	28	1.238190	0.000000	100	0.000000

Saved plot: /Users/tejasrao/Desktop/Master/Studies_at_Krea/Year 3/T9/CV/C
alibrationProject/plots/stage_6_3__mean_reprojection_error_with_sem.png



Plot Interpretation (Number of Views)

For this dataset, the mean reprojection error increases quickly from low n and then plateaus, rather than decreasing monotonically. This indicates that adding views introduces additional geometric/noise constraints that the linear pipeline balances at a stable compromise error.

Also, SEM is larger at low n , so subset composition matters more when very few views are used. SEM narrows as n grows, showing improved stability with more views.

Stage 6.4 (Extension): Plot of Relative Error of Intrinsic α_x vs Number of Views

This code block aims to find how the relative error of intrinsic-parameter estimation changes with increasing numbers of views.

Reference choice in this stage:

$$\alpha_x^{\text{ref}} = K_{11}^{\text{cv}},$$

where $\mathbf{K}^{\text{cv}}$ is the intrinsic matrix from OpenCV `cv.calibrateCamera`.

The code below computes relative errors for α_x per seed/subset, aggregates mean and SEM by n , and plots the error-bar curve.

```
In [26]: # Use OpenCV intrinsics as the reference for relative error.
K_ref_cv = K_opencv.copy()
alpha_x_ref = float(K_ref_cv[0, 0])

# Build per-seed alpha_x relative errors.
df_alpha_rel = df_stage6_seed_curves.copy()
df_alpha_rel["alpha_x_rel_error_pct"] = (
    100.0 * np.abs(df_alpha_rel["alpha_x"] - alpha_x_ref) / np.abs(alpha_x_ref)
)
```

```

# Aggregate mean and SEM by number of views for alpha_x.
df_alpha_x_summary = (
    df_alpha_rel.dropna(subset=["alpha_x_rel_error_pct"])
    .groupby("num_views")["alpha_x_rel_error_pct"]
    .agg(mean="mean", std="std", n="count")
    .reset_index()
    .sort_values("num_views")
)
df_alpha_x_summary["sem"] = df_alpha_x_summary["std"] / np.sqrt(df_alpha_

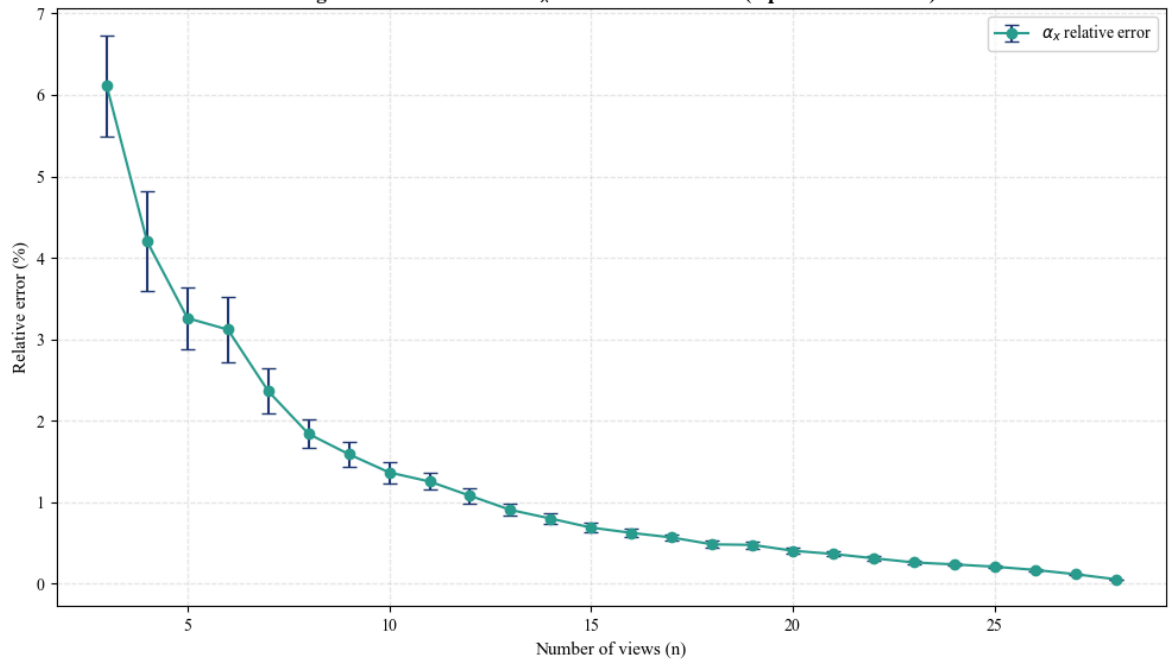
# Plot alpha_x relative-error trend with SEM.
plt.figure(figsize=(10, 6))
plt.errorbar(
    df_alpha_x_summary["num_views"],
    df_alpha_x_summary["mean"],
    yerr=df_alpha_x_summary["sem"],
    marker="o",
    capsize=4,
    label=r"$\alpha_x$ relative error",
    color=COLOUR_TEAL,
    ecolor=COLOUR_NAVY,
)
plt.xlabel("Number of views (n)")
plt.ylabel("Relative error (%)")
plt.title(
    r"Stage 6: Relative Error of $\alpha_x$ vs Number of Views (OpenCV Re
)
plt.grid(alpha=0.3)
plt.legend()
plt.tight_layout()
save_plot("Stage 6.4", "Relative error alpha_x with SEM")
plt.show()

display(
    df_alpha_x_summary.rename(
        columns={"mean": "alpha_x_mean_rel_error_pct", "sem": "alpha_x_se
    )
)

```

Saved plot: /Users/tejasrao/Desktop/Master/Studies_at_Krea/Year 3/T9/CV/C
alibrationProject/plots/stage_6_4__relative_error_alpha_x_with_sem.png

Stage 6: Relative Error of α_x vs Number of Views (OpenCV Reference)



	num_views	alpha_x_mean_rel_error_pct	std	n	alpha_x_sem
0	3	6.109213	6.035367	96	0.615982
1	4	4.200680	6.040319	97	0.613301
2	5	3.259853	3.779889	99	0.379893
3	6	3.119584	4.043526	100	0.404353
4	7	2.364407	2.755498	100	0.275550
5	8	1.839397	1.768491	100	0.176849
6	9	1.589386	1.480338	100	0.148034
7	10	1.366200	1.285611	100	0.128561
8	11	1.254658	1.028042	100	0.102804
9	12	1.080232	0.952855	100	0.095286
10	13	0.906715	0.724856	100	0.072486
11	14	0.799699	0.673447	100	0.067345
12	15	0.689505	0.557644	100	0.055764
13	16	0.624178	0.487238	100	0.048724
14	17	0.568010	0.430745	100	0.043075
15	18	0.483378	0.406091	100	0.040609
16	19	0.476941	0.414151	100	0.041415
17	20	0.405085	0.326710	100	0.032671
18	21	0.365763	0.289007	100	0.028901
19	22	0.312931	0.235577	100	0.023558
20	23	0.261675	0.210328	100	0.021033
21	24	0.237166	0.190397	100	0.019040
22	25	0.209256	0.148855	100	0.014886
23	26	0.169984	0.117792	100	0.011779
24	27	0.117283	0.082833	100	0.008283
25	28	0.055499	0.000000	100	0.000000

Plot Interpretation

The plotted relative error of intrinsic parameter α_x (with SEM) decreases as more views are used, with the strongest gains in lower n areas, as seen by the negative exponential type curve. This reproduces the qualitative trend reported by Zhang (Figure 2): intrinsic-parameter errors reduce as the number of model-plane images increases.

Stage 6.5 (Extension): Informed Sampling

I reasoned that if views with lower reprojection error images are used, then selecting them first should result in a monotonic increase in mean reprojection error with the increase in number of views. Following this, I also hypothesised that this could reduce calibration error at small n compared with random subsets.

This section aims to compare this quality-guided view-selection strategy (lowest per-view error first) against random selection trends. We call this **informed sampling** because view selection uses prior information (per-view reprojection error), rather than random choice.

Stage 6.5.1: Plot of Informed Sampling Error vs Number of Views

The code below ranks views by per-view reprojection error, builds growing best- n subsets, reruns calibration for each n , and plots mean reprojection error against number of views.

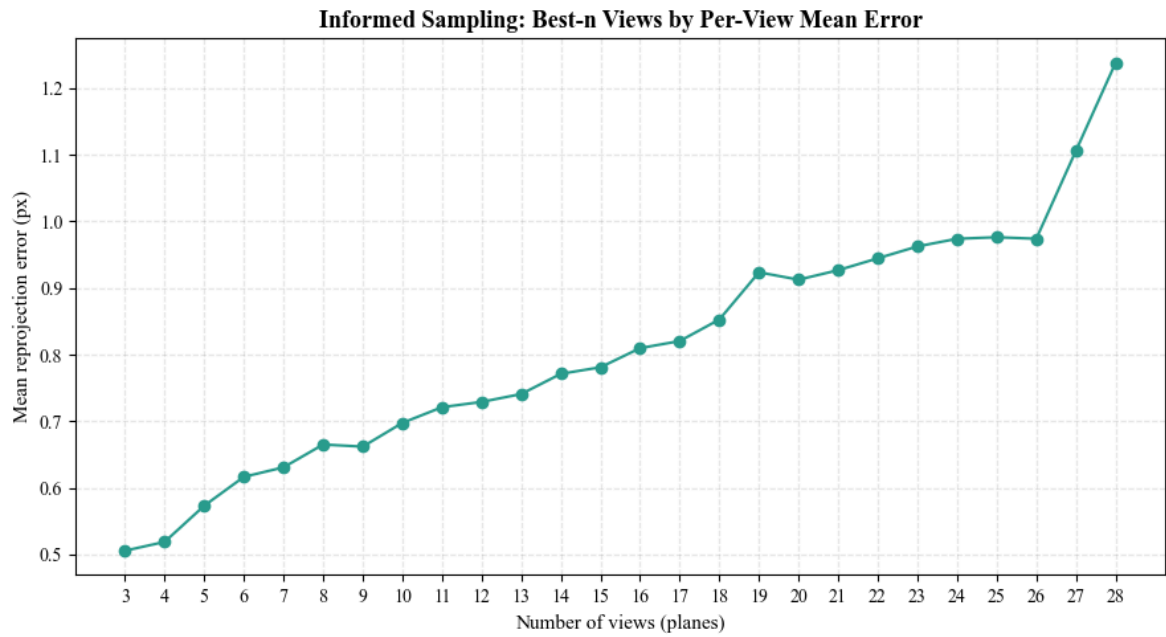
```
In [27]: # Informed sampling based on per-view mean reprojection error.
ranked_images = df_rep_error_fromP_perView.sort_values(
    "Mean Reprojection Error from P (px)"
)["Image"].tolist()
ranked_images = [
    (
        images_list_filtered[int(str(x).split()[1]) - 1]
        if str(x).startswith("View ")
        else x
    )
    for x in ranked_images
]

# Build cumulative best-n subsets and evaluate each one.
informed_rows = []
for n in range(3, len(ranked_images) + 1):
    subset_images = ranked_images[:n] # n views with least per-view mean
    out = run_full_calibration(subset_images, dataset_root=root)
    informed_rows.append(
        {
            "num_views": n,
            "mean_reprojection_error_px": out["overall_mean_reproj_error_
        }
    )
df_stage6_ablation_informed = pd.DataFrame(informed_rows)

# Plot informed-sampling error against number of views.
plt.figure(figsize=(9, 5))
plt.plot(
    df_stage6_ablation_informed["num_views"],
    df_stage6_ablation_informed["mean_reprojection_error_px"],
    marker="o",
    color=COLOUR_TEAL,
)
plt.xlabel("Number of views (planes)")
plt.ylabel("Mean reprojection error (px)")
plt.title("Informed Sampling: Best-n Views by Per-View Mean Error")
plt.grid(alpha=0.3)
plt.xticks(range(3, len(ranked_images) + 1))
plt.tight_layout()
save_plot("Stage 6.5.1", "Informed sampling reprojection error")
```

```
plt.show()  
df_stage6_ablation_informed
```

Saved plot: /Users/tejasrao/Desktop/Master/Studies_at_Krea/Year 3/T9/CV/C
alibrationProject/plots/stage_6_5_1__informed_sampling_reprojection_err
r.png



Out [27]:

	num_views	mean_reprojection_error_px
0	3	0.505583
1	4	0.518839
2	5	0.573188
3	6	0.616519
4	7	0.630763
5	8	0.665067
6	9	0.661896
7	10	0.697524
8	11	0.721120
9	12	0.729051
10	13	0.740873
11	14	0.771256
12	15	0.781056
13	16	0.809794
14	17	0.820461
15	18	0.852786
16	19	0.923705
17	20	0.912621
18	21	0.926882
19	22	0.944725
20	23	0.962548
21	24	0.974076
22	25	0.976348
23	26	0.974120
24	27	1.106227
25	28	1.238190

Plot Interpretation

The informed-sampling curve uses the lowest-error views first, so it often attains lower error for small n than random selection, as expected by the reasoning above. As n grows, the gap to random selection typically narrows because both strategies include a larger fraction of the available views.

Stage 6.5.2: Relative Error of Intrinsic α_x in Informed vs Randomised cases

This section compares informed sampling against random sampling using relative error in α_x at each number of views.

The code below computes and plots:

- informed-sampling relative error per n ,
- randomised mean and SEM over 100 seeds per n ,
- and their difference $\Delta_n = e_n^{\text{informed}} - e_n^{\text{random mean}}$.

```
In [28]: # Use the same random-sample size across comparison and testing.
seed_sample_size = 100

# Compute informed-sampling alpha_x relative error for each n.
informed_alpha_rows = []
for n in range(3, len(ranked_images) + 1):
    subset_images = ranked_images[:n]
    K_inf = run_full_calibration(subset_images, dataset_root=root)["K"]
    alpha_x_inf = float(K_inf[0, 0])
    alpha_x_rel_err_inf = 100.0 * abs(alpha_x_inf - alpha_x_ref) / abs(alpha_x_ref)
    informed_alpha_rows.append(
        {
            "num_views": n,
            "informed_alpha_x_rel_error_pct": alpha_x_rel_err_inf,
        }
    )

df_informed_alpha_x_rel = pd.DataFrame(informed_alpha_rows)

# Randomised baseline: mean + SEM alpha_x relative error using the same seed_sample_size
df_random_alpha_x_rel = (
    df_alpha_rel[df_alpha_rel["seed"] < seed_sample_size]
    .dropna(subset=["alpha_x_rel_error_pct"])
    .groupby("num_views")["alpha_x_rel_error_pct"]
    .agg(
        random_mean_alpha_x_rel_error_pct="mean",
        random_std_alpha_x_rel_error_pct="std",
        random_n="count",
    )
    .reset_index()
)
df_random_alpha_x_rel["random_sem_alpha_x_rel_error_pct"] = df_random_alpha_x_rel["random_std_alpha_x_rel_error_pct"] / np.sqrt(df_random_alpha_x_rel["random_n"])

# Merge and compare at each view count.
df_alpha_x_compare = (
    df_informed_alpha_x_rel.merge(df_random_alpha_x_rel, on="num_views",
    .sort_values("num_views")
    .reset_index(drop=True)
)
df_alpha_x_compare["delta_informed_minus_random_pct"] = (
    df_alpha_x_compare["informed_alpha_x_rel_error_pct"]
    - df_alpha_x_compare["random_mean_alpha_x_rel_error_pct"]
)

display(df_alpha_x_compare)

# Plot informed and randomised curves together.
```

```

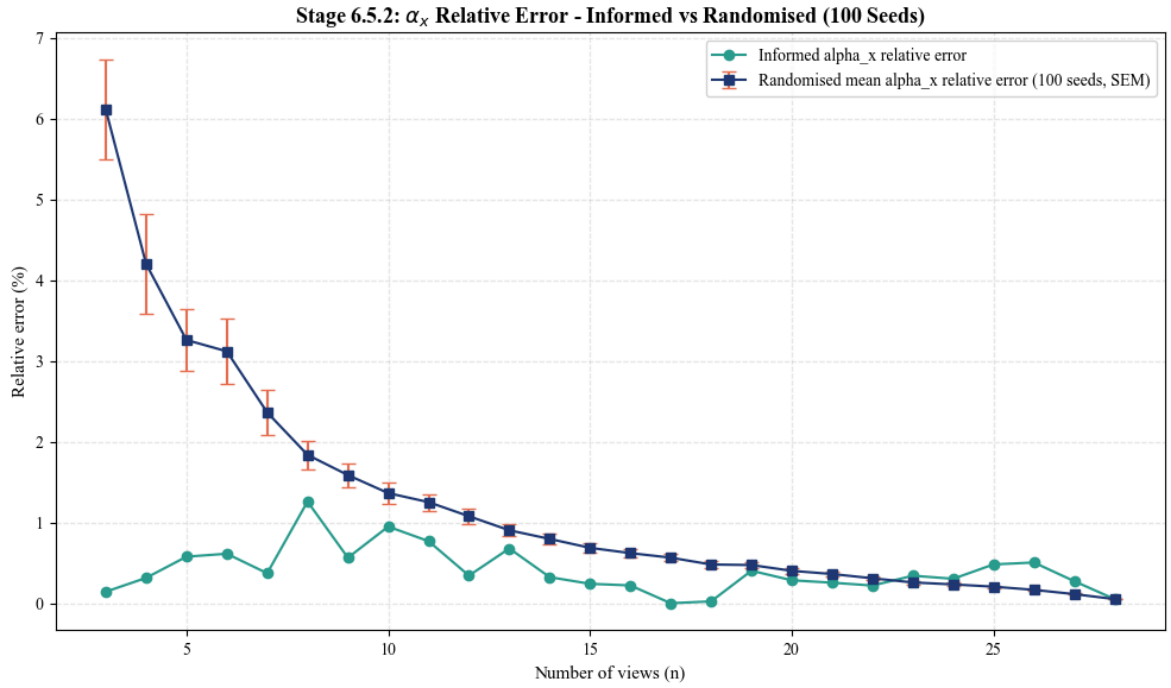
plt.figure(figsize=(10, 6))
plt.plot(
    df_alpha_x_compare["num_views"],
    df_alpha_x_compare["informed_alpha_x_rel_error_pct"],
    marker="o",
    label="Informed alpha_x relative error",
    color=COLOUR_TEAL,
)
plt.errorbar(
    df_alpha_x_compare["num_views"],
    df_alpha_x_compare["random_mean_alpha_x_rel_error_pct"],
    yerr=df_alpha_x_compare["random_sem_alpha_x_rel_error_pct"],
    marker="s",
    capsize=4,
    label=f"Randomised mean alpha_x relative error ({seed_sample_size} se",
    color=COLOUR_NAVY,
    ecolor=COLOUR_CORAL,
)
plt.xlabel("Number of views (n)")
plt.ylabel("Relative error (%)")
plt.title(
    rf"Stage 6.5.2: $\alpha_x$ Relative Error – Informed vs Randomised ({
)
plt.grid(alpha=0.3)
plt.legend()
plt.tight_layout()
save_plot("Stage 6.5.2", "Informed vs random alpha_x relative error")
plt.show()

# Plot delta to show the informed-vs-random gap by n.
plt.figure(figsize=(10, 4.5))
plt.plot(
    df_alpha_x_compare["num_views"],
    df_alpha_x_compare["delta_informed_minus_random_pct"],
    marker="d",
    color=COLOUR_PLUM,
)
plt.axhline(0.0, color="black", linestyle="--", linewidth=1)
plt.xlabel("Number of views (n)")
plt.ylabel(r"$\Delta_n$ (%)")
plt.title(
    rf"Stage 6.5.2: Delta (Informed – Randomised Mean) for $\alpha_x$ Rela
)
plt.grid(alpha=0.3)
plt.tight_layout()
save_plot("Stage 6.5.2", "Delta informed minus random alpha_x")
plt.show()

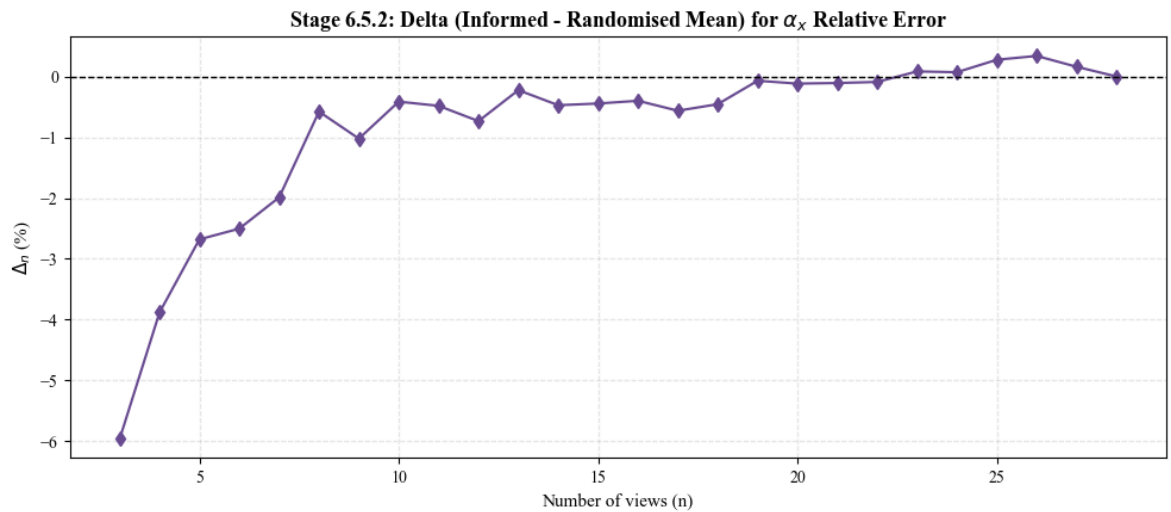
```

	num_views	informed_alpha_x_rel_error_pct	random_mean_alpha_x_rel_error_pct	random_st
0	3	0.146690	6.109213	
1	4	0.318184	4.200680	
2	5	0.580081	3.259853	
3	6	0.616595	3.119584	
4	7	0.375304	2.364407	
5	8	1.261661	1.839397	
6	9	0.567260	1.589386	
7	10	0.952361	1.366200	
8	11	0.771732	1.254658	
9	12	0.345998	1.080232	
10	13	0.681724	0.906715	
11	14	0.327190	0.799699	
12	15	0.244711	0.689505	
13	16	0.224660	0.624178	
14	17	0.004526	0.568010	
15	18	0.027499	0.483378	
16	19	0.406548	0.476941	
17	20	0.288176	0.405085	
18	21	0.258459	0.365763	
19	22	0.223071	0.312931	
20	23	0.344886	0.261675	
21	24	0.306944	0.237166	
22	25	0.484619	0.209256	
23	26	0.508085	0.169984	
24	27	0.276789	0.117283	
25	28	0.055499	0.055499	

Saved plot: /Users/tejasrao/Desktop/Master/Studies_at_Krea/Year 3/T9/CV/C
alibrationProject/plots/stage_6_5_2__informed_vs_random_alpha_x_relative_
error.png



Saved plot: /Users/tejasrao/Desktop/Master/Studies_at_Krea/Year 3/T9/CV/C alibrationProject/plots/stage_6_5_2_delta_informed_minus_random_alpha_x.png



Plot Interpretation

The first plot compares informed sampling with the randomised mean (with SEM) across view counts. Lower informed values indicate improved intrinsic-error behaviour relative to random sampling, which corroborates the hypothesis stated above.

The second plot shows this gap directly: Given that $\Delta_n = e_n^{\text{informed}} - e_n^{\text{random mean}}$, an initial trend $\Delta_n < 0$ shows that informed sampling outperforms random sampling. For larger number of views, $\Delta_n \rightarrow 0$, meaning that informed sampling performs similarly to random sampling.

Stage 6.5.3: One-Sided Hypothesis Test Across View Counts

This section tests whether random sampling has higher expected relative error than informed sampling at each view count n .

Hypotheses (per n):

- $H_0 : \mu_n^{\text{random}} \leq e_n^{\text{informed}}$
- $H_1 : \mu_n^{\text{random}} > e_n^{\text{informed}}$

Test setup:

- one-sample, one-sided t-test,
- FPR threshold $\alpha = 0.01$,
- random-sampling size linked to Stage 6.5.2 (`seed_sample_size = 100`).

We use the Pearson Correlation coefficient to quantify the direction and strength of trend with number of views, rather than relying only on visual inspection. In the later log-scale hypothesis-test analysis, correlation between n and $\log_{10}(p_n)$ checks whether evidential strength changes systematically as n increases.

Note: An LLM was used for to synthesise the code snippet for hypothesis testing below, and the logic was verified post generation. The underlying idea, hypothesis and plotting are original.

```
In [29]: from scipy import stats

# Compute per-n one-sided tests and collect p-values.

alpha_fpr = 0.01

common_n = sorted(
    set(df_informed_alpha_x_rel["num_views"]).intersection(
        set(df_alpha_rel.loc[df_alpha_rel["seed"] < seed_sample_size, "num_views"])
    )
)

df_alpha_x_ttest = pd.DataFrame(
    {
        "num_views": common_n,
        "informed_alpha_x_rel_error_pct": [
            float(
                df_informed_alpha_x_rel.loc[
                    df_informed_alpha_x_rel["num_views"] == n,
                    "informed_alpha_x_rel_error_pct",
                ].iloc[0]
            )
            for n in common_n
        ],
    }
)

df_alpha_x_ttest["random_mean_alpha_x_rel_error_pct"] = [
    float(
        df_alpha_rel[
            (df_alpha_rel["seed"] < seed_sample_size) & (df_alpha_rel["num_views"] == n)
        ].iloc[0]["alpha_x_rel_error_pct"]
    )
    for n in common_n
]

df_alpha_x_ttest["n_random"] = [
    int(
```

```

        df_alpha_rel[
            (df_alpha_rel["seed"] < seed_sample_size) & (df_alpha_rel["num_views"] > 1)
        ]["alpha_x_rel_error_pct"]
        .dropna()
        .shape[0]
    )
    for n in common_n
]
df_alpha_x_ttest[["t_stat", "p_value"]] = [
    stats.ttest_1samp(
        df_alpha_rel[
            (df_alpha_rel["seed"] < seed_sample_size) & (df_alpha_rel["num_views"] > 1)
        ]["alpha_x_rel_error_pct"]
        .dropna()
        .values,
        popmean=float(
            df_informed_alpha_x_rel.loc[
                df_informed_alpha_x_rel["num_views"] == n,
                "informed_alpha_x_rel_error_pct",
            ].iloc[0]
        ),
        alternative="greater",
    )
    for n in common_n
]
df_alpha_x_ttest["reject_H0_at_FPR_0_01"] = df_alpha_x_ttest["p_value"] < alpha_fpr

display(df_alpha_x_ttest)

# Fit a straight line in log10(p)-space for trend inspection.

x_vals = df_alpha_x_ttest["num_views"].to_numpy(dtype=float)
y_log10 = np.log10(df_alpha_x_ttest["p_value"].to_numpy(dtype=float))

slope, intercept = np.polyfit(x_vals, y_log10, 1)
r_val, p_r_one_sided = stats.pearsonr(x_vals, y_log10, alternative="greater")
r_positive_significant = bool((r_val > 0) and (p_r_one_sided < alpha_fpr))

x_line = np.linspace(x_vals.min(), x_vals.max(), 200)
y_line = 10 ** (slope * x_line + intercept)

# Plot p-values on log scale with the fitted trend line.
plt.figure(figsize=(10, 5))
plt.plot(
    df_alpha_x_ttest["num_views"],
    df_alpha_x_ttest["p_value"],
    marker="o",
    label="p-value (one-sided)",
    color=COLOUR_NAVY,
)
plt.plot(
    x_line,
    y_line,
    linestyle="--",
    linewidth=2,
    color=COLOUR_CORAL,
    label=r"Log-space regression fit",
)
plt.axhline(alpha_fpr, color=COLOUR_CRIMSON, linestyle="--", label=f"FPR")
plt.xlabel("Number of views (n)")

```

```

plt.ylabel("p-value")
plt.title(
    rf"Stage 6.5.3: Per-n one-sided t-test p-values for $\alpha_x$ Relati
)
plt.grid(alpha=0.3)
plt.yscale("log")
plt.text(
    0.02,
    0.02,
    rf"r = {r_val:.3f} and significant @ FPR={alpha_fpr}",
    transform=plt.gca().transAxes,
    fontsize=9,
    bbox=dict(facecolor="white", alpha=0.8, edgecolor="none"),
)
plt.legend(loc="lower right")

plt.tight_layout()
save_plot("Stage 6.5.3", "One-sided t-test p-values log scale")
plt.show()

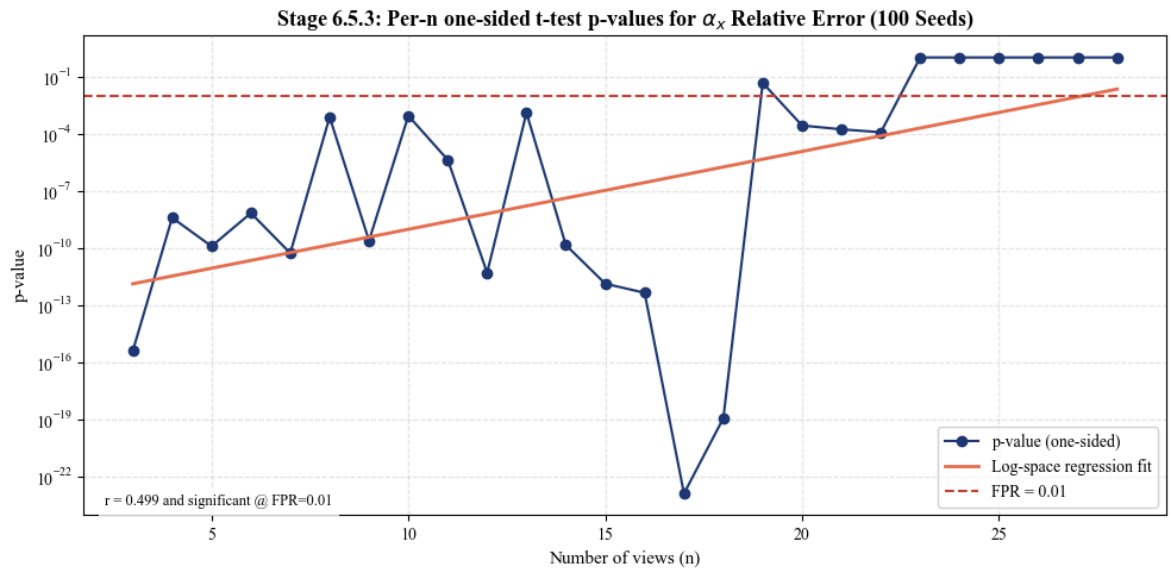
```

/Users/tejasrao/anaconda3/envs/IIT-D_QCML/lib/python3.12/site-packages/scipy/stats/_axis_nan_policy.py:592: RuntimeWarning: Precision loss occurred in moment calculation due to catastrophic cancellation. This occurs when the data are nearly identical. Results may be unreliable.

```
res = hypotest_fun_out(*samples, **kwds)
```

	num_views	informed_alpha_x_rel_error_pct	random_mean_alpha_x_rel_error_pct	n_random
0	3	0.146690	6.109213	96
1	4	0.318184	4.200680	97
2	5	0.580081	3.259853	99
3	6	0.616595	3.119584	100
4	7	0.375304	2.364407	100
5	8	1.261661	1.839397	100
6	9	0.567260	1.589386	100
7	10	0.952361	1.366200	100
8	11	0.771732	1.254658	100
9	12	0.345998	1.080232	100
10	13	0.681724	0.906715	100
11	14	0.327190	0.799699	100
12	15	0.244711	0.689505	100
13	16	0.224660	0.624178	100
14	17	0.004526	0.568010	100
15	18	0.027499	0.483378	100
16	19	0.406548	0.476941	100
17	20	0.288176	0.405085	100
18	21	0.258459	0.365763	100
19	22	0.223071	0.312931	100
20	23	0.344886	0.261675	100
21	24	0.306944	0.237166	100
22	25	0.484619	0.209256	100
23	26	0.508085	0.169984	100
24	27	0.276789	0.117283	100
25	28	0.055499	0.055499	100

Saved plot: /Users/tejasrao/Desktop/Master/Studies_at_Krea/Year_3/T9/CV/C
alibrationProject/plots/stage_6_5_3__one_sided_t_test_p_values_log_scale.
png



Plot Interpretation

On the log-scale p-value plot, the fitted regression line summarises whether p-values tend to increase or decrease with the number of views. As the slope is positive (i.e., $r \approx 0.5$), it indicates p-values tend to increase with n , meaning evidence against H_0 increase as more views are included, i.e., for smaller subsets, informed sampling yields better results than random sampling.

Note: Many per- n hypothesis tests are conducted here (though at a low false positive rate), meaning we need to interpret individual p-values with caution. However, the general trend is still an increase, meaning that we can still make the interpretation above.